최신 데이터 분석

NetApp Solutions

NetApp
February 02, 2022

This PDF was generated from https://docs.netapp.com/ko-kr/netapp-solutions/data-analytics/confluent-kafka-introduction.html on February 02, 2022. Always check docs.netapp.com for the latest.

목차

NetApp의 최신 데이터 분석 솔루션 1
Confluent Kafka 모범 사례 1
NetApp 하이브리드 클라우드 데이터 솔루션 - 고객 사용 사례를 기반으로 Spark 및 Hadoop 29

NetApp의 최신 데이터 분석 솔루션

Confluent Kafka 모범 사례

TR-4912: NetApp을 통해 Confluent Kafka 계층형 스토리지를 위한 모범 사례 지침

Karthkeyan Nagalingam, Joseph Kandatillarambil, NetApp Rankesh Kumar, Confluent

Apache Kafka는 매일 수조 건의 이벤트를 처리할 수 있는 커뮤니티 분산 이벤트 스트리밍 플랫폼입니다. 처음에 메시징 큐로 구상된 Kafka는 분산 커밋 로그의 추상화를 기반으로 합니다. Kafka는 2011년 LinkedIn에서 제작 및 오픈 소스를 통해 메시지 큐에서 완전한 이벤트 스트리밍 플랫폼으로 발전했습니다. Confluent는 Apache Kafka를 Confluent Platform과 함께 배포할 수 있도록 합니다. Confluent Platform은 Kafka를 보완하는 추가적인 커뮤니티 및 상용 기능을 제공하여 대규모 생산 환경에서 운영자와 개발자 모두의 스트리밍 경험을 개선하도록 설계되었습니다.

이 문서에서는 다음 내용을 제공하여 NetApp의 오브젝트 스토리지 제품에서 Confluent Tiered Storage를 사용하기 위한 모범 사례 지침을 설명합니다.

- NetApp 오브젝트 스토리지를 위한 Confluent 검증 – NetApp StorageGRID
- 계층형 스토리지 성능 테스트
- NetApp 스토리지 시스템에 대한 Confluent의 모범 사례 지침

ContFluent Tiered Storage를 선택해야 하는 이유

Confluent는 많은 애플리케이션, 특히 빅데이터, 분석 및 스트리밍 워크로드를 위한 기본 실시간 스트리밍 플랫폼이 되었습니다. 계층적 스토리지를 사용하면 사용자가 Confluent 플랫폼의 스토리지에서 컴퓨팅을 분리할 수 있습니다. Data Fabric을 활용하면 데이터를 보다 경제적으로 저장하고, 사실상 무한대의 데이터를 저장하고, 필요 시 워크로드를 스케일업(또는 스케일다운) 할 수 있으며, 데이터 및 테넌트 재조정과 같은 관리 작업을 더 쉽게 수행할 수 있습니다. S3 호환 스토리지 시스템은 이러한 모든 기능을 활용하여 모든 이벤트의 데이터를 한 곳에서 대중화할 수 있으며, 복잡한 데이터 엔지니어링이 필요하지 않습니다. Kafka에 계층형 스토리지를 사용해야 하는 이유에 대한 자세한 내용은 을 참조하십시오 "이 기사는 Confluent에 의해 작성되었습니다".

NetApp StorageGRID를 계층형 스토리지로 선택해야 하는 이유

StorageGRID은 NetApp에서 제공하는 업계 최고의 오브젝트 스토리지 플랫폼입니다. StorageGRID은 Amazon S3(Simple Storage Service) API를 비롯한 업계 표준 오브젝트 API를 지원하는 소프트웨어 정의 오브젝트 기반 스토리지 솔루션입니다. StorageGRID는 비정형 데이터를 대규모로 저장 및 관리하여 안전하고 내구성 있는 오브젝트 스토리지를 제공합니다. 콘텐츠가 적절한 위치, 적합한 시간 및 적합한 스토리지 계층에 배치되어 전 세계적으로 분산된 다양한 미디어의 워크플로우를 최적화하고 비용을 줄입니다.

StorageGRID의 가장 큰 차별화 요소는 정책 기반 데이터 라이프사이클 관리를 지원하는 ILM(정보 라이프사이클 관리) 정책 엔진입니다. 정책 엔진은 메타데이터를 사용해 수명 주기 동안 데이터가 저장되는 방식을 관리하여 처음에 성능을 최적화하고 데이터 사용 기간에 따라 비용 및 내구성을 자동으로 최적화할 수 있습니다.

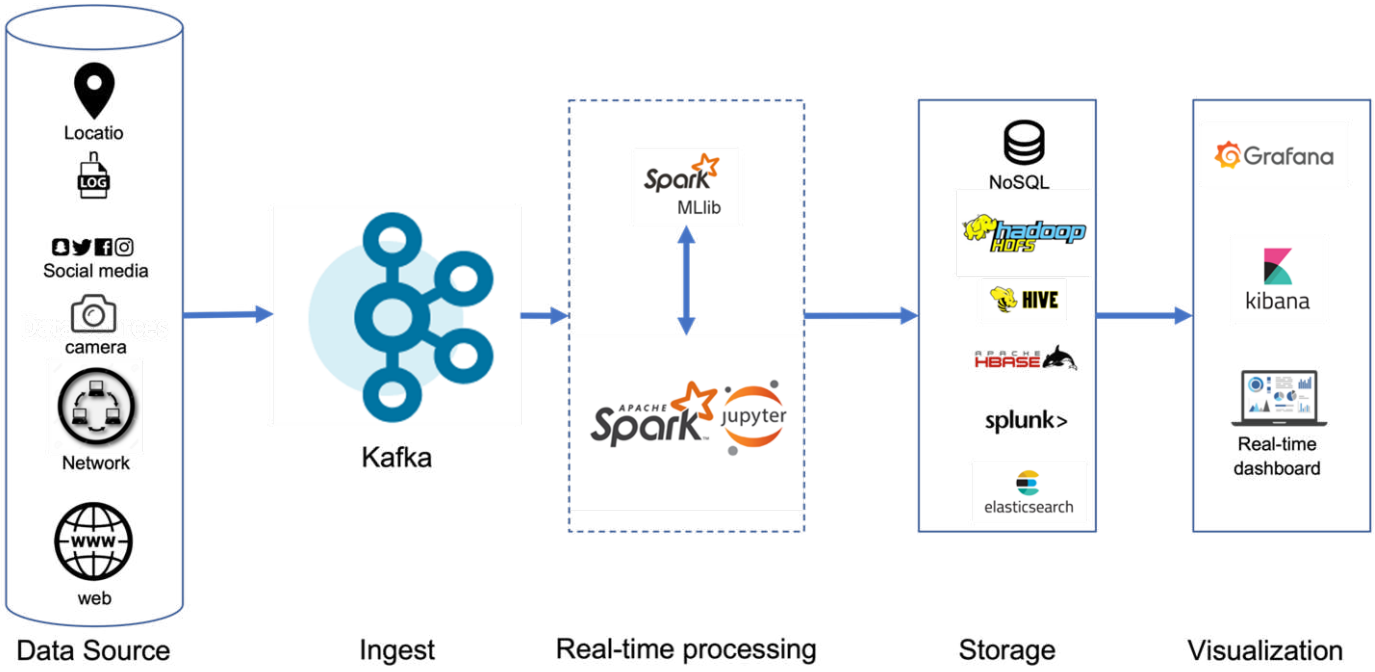
ContFluent Tiered Storage 활성화

계층형 스토리지의 기본 개념은 데이터 스토리지와 데이터 처리 작업을 분리하는 것입니다. 이와 같은 분리 덕분에 데이터 스토리지 계층과 데이터 처리 계층이 독립적으로 확장하는 것이 훨씬 쉬워졌습니다.

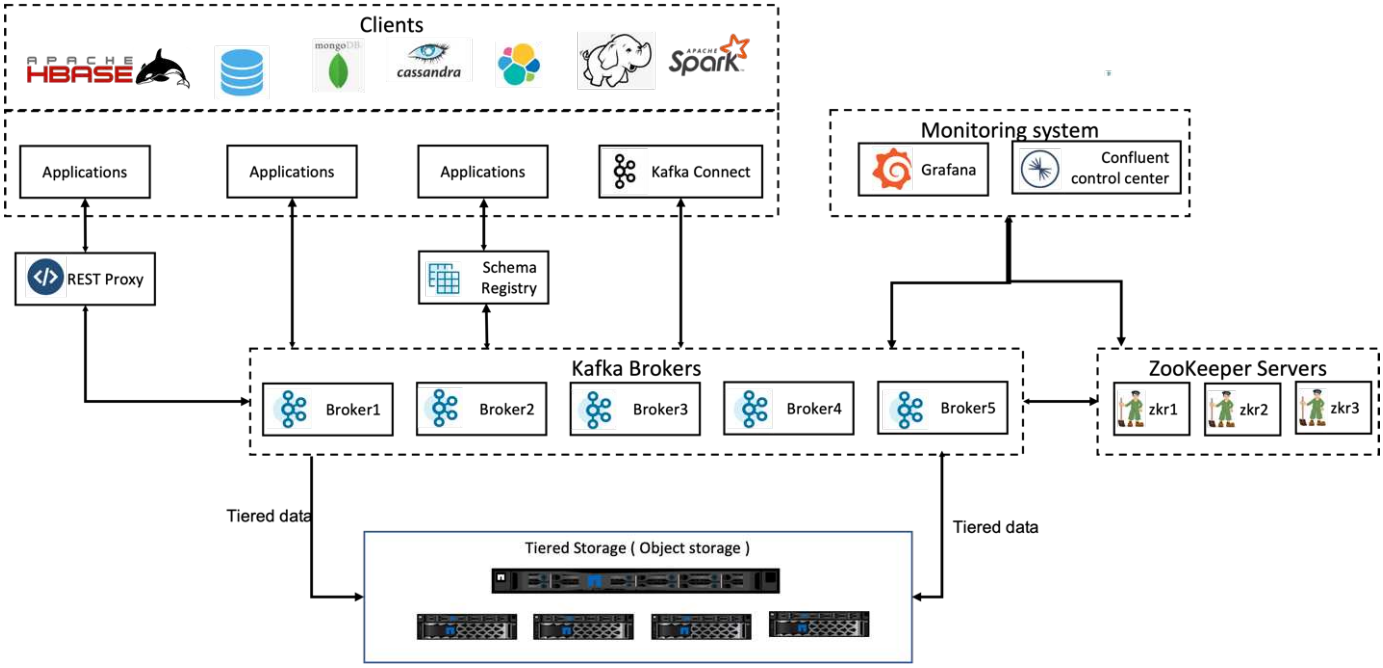
Confluent를 위한 계층형 스토리지 솔루션은 두 가지 요소를 고려해야 합니다. 먼저, 목록 작업의 일관성 오류, 간헐적인 개체 가용성 손실 등과 같은 일반적인 개체 저장소의 일관성 및 가용성 속성을 문제를 해결 또는 방지해야 합니다. 두

번째로, 이 제품은 계층형 스토리지와 Kafka의 복제 및 내결함성 모델 간의 상호 작용을 올바르게 처리해야 하며, 이러한 상호 작용에는 좀비 리더가 계속해서 오프셋 범위를 계층화할 수 있는 가능성이 포함됩니다. NetApp 오브젝트 스토리지는 일관된 오브젝트 가용성과 HA 모델을 모두 제공하므로 피로한 스토리지를 계층 오프셋 범위에 사용할 수 있습니다. NetApp 오브젝트 스토리지는 일관된 오브젝트 가용성과 HA 모델을 제공하여 지친 스토리지를 계층 오프셋 범위에 사용할 수 있도록 합니다.

계층형 스토리지를 사용하면 스트리밍 데이터의 지연 시간이 짧은 읽기 및 쓰기 작업에 고성능 플랫폼을 사용할 수 있으며, 처리량이 높은 내역 읽기를 위해 NetApp StorageGRID와 같은 경제적이고 확장 가능한 오브젝트 저장소를 사용할 수도 있습니다. NetApp은 Spark with NetApp 스토리지 컨트롤러 및 세부 정보를 위한 기술 솔루션도 갖추고 있습니다. 다음 그림은 Kafka가 실시간 분석 파이프라인에 어떻게 부합하는지를 보여줍니다.



다음 그림은 NetApp StorageGRID가 Confluent Kafka의 오브젝트 스토리지 계층으로 적합한 방식을 보여줍니다.



"다음은 솔루션 아키텍처 세부 정보입니다."

솔루션 아키텍처 세부 정보

"이전: 소개."

이 섹션에서는 Confluent 검증에 사용된 하드웨어 및 소프트웨어에 대해 다룹니다. 이 정보는 NetApp 스토리지를 사용하는 Confluent Platform 구축에 적용할 수 있습니다. 다음 표에서는 테스트를 거친 솔루션 아키텍처 및 기본 구성 요소에 대해 설명합니다.

솔루션 구성 요소	세부 정보
Confluent Kafka 버전 6.2	• 세 명의 주키퍼입니다 • 브로커 서버 5대 • 5개의 도구 서버 • 하나의 Grafana • 제어 센터 1개
Linux(Ubuntu 18.04)	모든 서버
계층형 스토리지를 위한 NetApp StorageGRID	• StorageGRID 소프트웨어 • SG1000(로드 밸런서) 1개 • SGF6024 4개 • 24 x 800 SSD 4개 • S3 프로토콜 • 4 x 100GbE(브로커와 StorageGRID 인스턴스 간 네트워크 연결)
15 Fujitsu Primergy RX2540 서버	각 장착 사양: * CPU 2개, 물리적 코어 16개 * Intel Xeon * 256GB 물리적 메모리 * 100GbE 듀얼 포트

"다음: 기술 개요"

기술 개요

"이전: 솔루션 아키텍처 세부 정보."

이 섹션에서는 이 솔루션에 사용된 기술에 대해 설명합니다.

NetApp StorageGRID를 참조하십시오

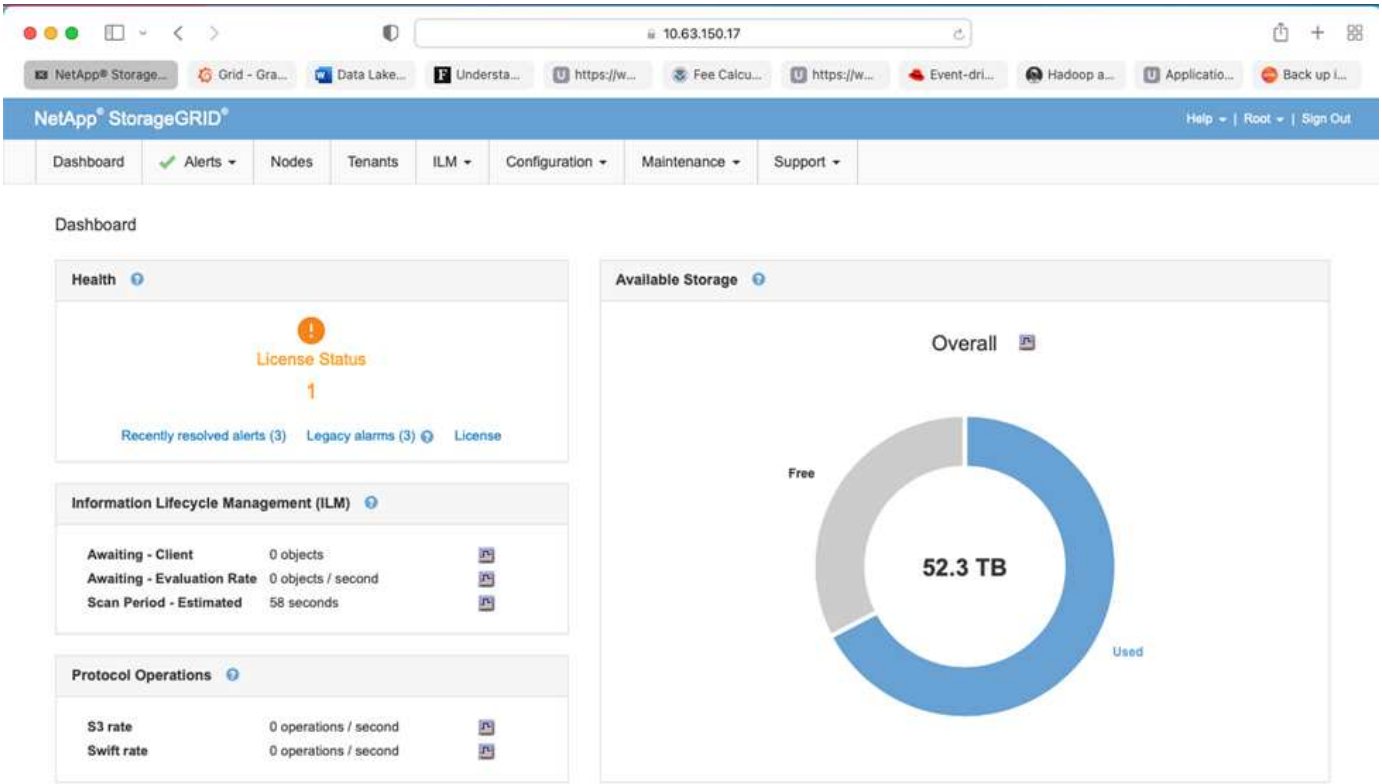
NetApp StorageGRID는 비용 효율적인 고성능 오브젝트 스토리지 플랫폼입니다. 계층형 스토리지를 사용하면 중개인의 로컬 스토리지 또는 SAN 스토리지에 저장된 Confluent Kafka의 데이터 대부분이 원격 오브젝트 저장소로 오프로드됩니다. 이 구성은 클러스터 재조정, 확장 또는 축소 또는 실패한 브로커 교체에 드는 시간과 비용을 줄여 운영 효율성을 크게 개선합니다. 오브젝트 스토리지는 오브젝트 저장소 계층에 있는 데이터를 관리하는 데 중요한 역할을 합니다. 따라서 적합한 오브젝트 스토리지를 선택하는 것이 중요합니다.

StorageGRID는 분산된 노드 기반 그리드 아키텍처를 사용하여 정책 중심의 지능형 글로벌 데이터 관리를 제공합니다. 정교한 데이터 관리 기능과 결합된 유비쿼터스 글로벌 오브젝트 네임스페이스를 통해 페타바이트 단위의 비정형 데이터와 수십억 개의 오브젝트 관리를 간소화합니다. 단일 호출 개체 액세스는 사이트 간에 확장되고 고가용성 아키텍처를 단순화하는 동시에 사이트 또는 인프라 중단과 관계없이 지속적인 개체 액세스를 보장합니다.

멀티 테넌시를 사용하면 동일한 그리드 내에서 여러 비정형 클라우드 및 엔터프라이즈 데이터 애플리케이션을 안전하게 서비스할 수 있으므로 NetApp StorageGRID의 ROI 및 사용 사례가 증가합니다. 여러 지역에서 내구성, 보호, 성능, 인접성을 최적화하여 메타데이터 기반 오브젝트 라이프사이클 정책을 통해 여러 서비스 레벨을 생성할 수 있습니다. 사용자는 데이터 관리 정책을 조정하고 트래픽 제한을 모니터링 및 적용하여 끊임없이 변화하는 IT 환경에서 요구 사항이 변경됨에 따라 데이터 환경을 중단 없이 다시 조정할 수 있습니다.

Grid Manager를 통한 간편한 관리

StorageGRID 그리드 관리자는 브라우저 기반의 그래픽 인터페이스로, 단일 창에서 전세계에 분산된 위치에 걸쳐 StorageGRID 시스템을 구성, 관리 및 모니터링할 수 있습니다.



StorageGRID 그리드 관리자 인터페이스를 사용하여 다음 작업을 수행할 수 있습니다.

- 이미지, 비디오, 레코드 등 전 세계에 분산된 페타바이트 규모의 오브젝트 저장소를 관리합니다.
- 그리드 노드 및 서비스를 모니터링하여 개체 가용성을 보장합니다.
- ILM(정보 수명 주기 관리) 규칙을 사용하여 시간이 지남에 따라 오브젝트 데이터의 배치를 관리합니다. 이러한 규칙은 수집된 개체의 데이터, 데이터가 손실되지 않도록 보호하는 방법, 오브젝트 데이터가 저장되는 위치 및 기간에 대해 적용됩니다.
- 시스템 내의 트랜잭션, 성능 및 운영을 모니터링합니다.

정보 수명 주기 관리 정책

StorageGRID는 오브젝트의 복제본 보존 및 특정 성능 및 데이터 보호 요구사항에 따라 오브젝트를 저장할 수 있도록

2+1 및 4+2(특히 다른)와 같은 EC(삭제 코딩) 스키마를 사용하는 등의 유연한 데이터 관리 정책을 제공합니다. 시간에 따라 워크로드와 요구사항이 달라지날수록 ILM 정책도 시간에 따라 바뀌어야 합니다. ILM 정책을 수정하는 것은 핵심 기능이므로 StorageGRID 고객은 끊임없이 변화하는 환경에 빠르고 쉽게 적응할 수 있습니다. 를 확인하십시오 "ILM 정책" 및 StorageGRID에서 설정.

성능

StorageGRID는 VM, 베어 메탈 또는 특수 제작된 어플라이언스 등 스토리지 노드를 추가하여 성능을 확장합니다 "SG5712, SG5760, SG6060 또는 SGF6024". 이 테스트에서는 SGF6024 어플라이언스를 사용하는 최소 크기의 3노드 그리드로 Apache Kafka의 주요 성능 요구사항을 초과했습니다. 고객이 추가 브로커로 Kafka 클러스터를 확장함에 따라 스토리지 노드를 추가하여 성능과 용량을 확장할 수 있습니다.

부하 분산 장치 및 엔드포인트 구성

StorageGRID의 관리 노드는 StorageGRID 시스템을 보고, 구성하고, 관리할 수 있는 그리드 관리자 UI(사용자 인터페이스) 및 REST API 엔드포인트와 시스템 작업을 추적할 수 있는 감사 로그를 제공합니다. Confluent Kafka 계층형 스토리지에 가용성이 높은 S3 엔드포인트를 제공하기 위해 관리 노드와 게이트웨이 노드에서 서비스로 실행되는 StorageGRID 로드 밸런서를 구현했습니다. 또한 로드 밸런서는 로컬 트래픽을 관리하고 GSLB(Global Server Load Balancing)에 연결하여 재해 복구를 지원합니다.

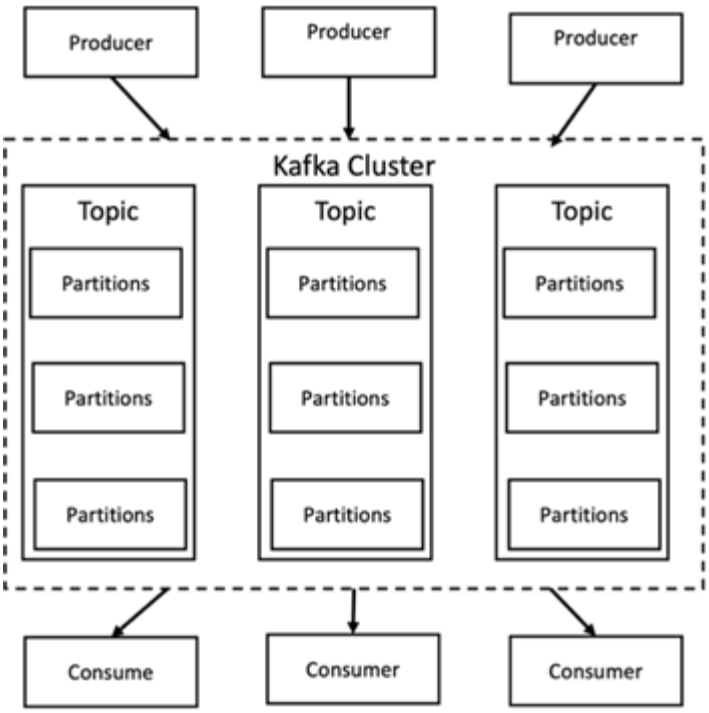
엔드포인트 구성을 더욱 개선하기 위해 StorageGRID는 관리 노드에 내장된 트래픽 분류 정책을 제공하고, 워크로드 트래픽을 모니터링하고, 워크로드에 다양한 QoS(서비스 품질) 제한을 적용할 수 있도록 지원합니다. 트래픽 분류 정책은 게이트웨이 노드 및 관리 노드에 대한 StorageGRID 부하 분산 서비스의 끝점에 적용됩니다. 이러한 정책은 트래픽 제한 및 모니터링을 지원할 수 있습니다.

StorageGRID의 트래픽 분류

StorageGRID에는 QoS 기능이 내장되어 있습니다. 트래픽 분류 정책은 클라이언트 애플리케이션에서 들어오는 다양한 유형의 S3 트래픽을 모니터링하는 데 도움이 될 수 있습니다. 그런 다음 정책을 생성하여 적용하여 In/Out 대역폭, 읽기/쓰기 동시 요청 수 또는 읽기/쓰기 요청 속도에 따라 이 트래픽에 제한을 적용할 수 있습니다.

아파치 카프카

Apache Kafka는 Java 및 Scala로 작성된 스트림 처리를 사용하는 소프트웨어 버스의 프레임워크입니다. 이 제품은 실시간 데이터 피드 처리를 위한 높은 처리량의 짧은 대기 시간을 갖춘 통합 플랫폼을 제공하기 위해 마련되었습니다. Kafka는 데이터 내보내기 및 가져오기를 위해 Kafka Connect를 통해 외부 시스템에 연결할 수 있으며 Java 스트림 처리 라이브러리인 Kafka 스트림을 제공합니다. Kafka는 효율성을 위해 최적화된 바이너리 TCP 기반 프로토콜을 사용하며, 네트워크를 통한 왕복 작업의 오버헤드를 줄이기 위해 자연스럽게 메시지를 그룹화하는 "메시지 세트" 추상화에 의존합니다. 이렇게 하면 순차 디스크 작업, 더 큰 네트워크 패킷 및 연속 메모리 블록이 증가하므로 Kafka는 랜덤 메시지 쓰기의 폭주 스트림을 선형 쓰기로 전환할 수 있습니다. 다음 그림은 Apache Kafka의 기본 데이터 흐름을 보여 줍니다.



Kafka는 생산자라는 임의의 수의 프로세스에서 가져온 키 값 메시지를 저장합니다. 데이터는 여러 주제 내의 서로 다른 파티션으로 분할될 수 있습니다. 파티션 내에서 메시지는 해당 오프셋(파티션 내의 메시지 위치)에 의해 엄격하게 정렬되고 타임 스탬프와 함께 인덱싱되고 저장됩니다. 소비자라고 하는 다른 프로세스는 파티션에서 메시지를 읽을 수 있습니다. 스트림 처리를 위해 Kafka는 Kafka의 데이터를 사용하는 Java 애플리케이션을 작성하고 결과를 Kafka에 다시 쓸 수 있는 스트림 API를 제공합니다. Apache Kafka는 Apache Apex, Apache Flink, Apache Spark, Apache Storm, Apache nifi 등의 외부 스트림 처리 시스템과도 작동합니다.

Kafka는 하나 이상의 서버(브로커)로 구성된 클러스터에서 실행되며 모든 항목의 파티션이 클러스터 노드에 분산됩니다. 또한 파티션이 여러 브로커에 복제됩니다. 이 아키텍처를 통해 Kafka는 내결함성이 있는 방식으로 대규모 메시지 스트림을 전달할 수 있으며 JMS(Java Message Service), AMQP(Advanced Message Queuing Protocol) 등의 기존 메시징 시스템을 대체할 수 있습니다. Kafka는 0.11.0.0 릴리스 이후 트랜잭션 쓰기를 제공하여 스트림 API를 사용하여 정확히 한 번의 스트림 처리를 제공합니다.

Kafka는 일반과 컴팩션이라는 두 가지 유형의 주제를 지원합니다. 정규 주제는 보존 시간 또는 공간 바인딩으로 구성할 수 있습니다. 지정된 보존 시간보다 오래된 레코드가 있거나 파티션에 대해 바인딩된 공간이 초과된 경우 Kafka는 사용 가능한 저장소 공간을 확보하기 위해 이전 데이터를 삭제할 수 있습니다. 기본적으로 항목은 보존 기간이 7일로 구성되지만 데이터를 무기한 저장할 수도 있습니다. 압축된 항목의 경우 시간 또는 공간 범위에 따라 레코드가 만료되지 않습니다. 대신 Kafka는 나중에 받은 메시지를 동일한 키를 사용하는 이전 메시지의 업데이트로 취급하며 키당 최신 메시지를 삭제하지 않도록 보장합니다. 사용자는 특정 키에 대해 null 값을 갖는 소위 tombstone 메시지를 작성하여 메시지를 완전히 삭제할 수 있습니다.

Kafka에는 다음과 같은 5가지 주요 API가 있습니다.

- * Producer API. * 응용 프로그램에서 레코드 스트림을 게시할 수 있도록 허용합니다.
- * 소비자 API. * 응용 프로그램에서 항목 및 레코드 스트림 프로세스를 구독할 수 있도록 허용합니다.
- * Connector API. * 항목을 기존 응용 프로그램에 연결할 수 있는 재사용 가능한 프로듀서 및 소비자 API를 실행합니다.
- * Streams API. * 이 API는 입력 스트림을 출력으로 변환하고 결과를 생성합니다.
- * Admin API. * Kafka 주제, 브로커 및 기타 Kafka 객체를 관리하는 데 사용됩니다.

소비자 및 생산자 API는 Kafka 메시징 프로토콜을 기반으로 하며 Java의 Kafka 소비자 및 생산자 클라이언트에 대한 참조 구현을 제공합니다. 기본 메시징 프로토콜은 개발자가 프로그래밍 언어로 소비자 또는 생산자 클라이언트를 작성하는 데 사용할 수 있는 이진 프로토콜입니다. 이렇게 하면 JVM(Java Virtual Machine) 에코시스템에서 Kafka가 잠금 해제됩니다. 사용 가능한 비 Java 클라이언트 목록은 Apache Kafka wiki에서 유지 관리됩니다.

Apache Kafka 사용 사례

Apache Kafka는 메시징, 웹 사이트 활동 추적, 메트릭, 로그 집계, 스트림 처리, 이벤트 소싱 및 로깅 커밋

- Kafka는 향상된 처리량, 내장 파티셔닝, 복제 및 내결함성 기능을 제공하므로 대규모 메시지 처리 애플리케이션에 적합한 솔루션입니다.
- Kafka는 실시간 게시 구독 피드 집합으로 추적 파이프라인에서 사용자의 활동(페이지 보기, 검색)을 재구축할 수 있습니다.
- Kafka는 운영 모니터링 데이터에 자주 사용됩니다. 이를 위해서는 분산된 애플리케이션에서 통계를 집계하여 운영 데이터의 중앙 집중식 피드를 생성하는 작업이 필요합니다.
- 많은 사람들이 Kafka를 로그 집계 솔루션의 대안으로 사용합니다. 로그 집계는 일반적으로 서버에서 물리적 로그 파일을 수집하여 처리를 위해 중앙 위치(예: 파일 서버 또는 HDFS)에 배치합니다. Kafka는 파일 세부 정보를 추상화하고 로그 또는 이벤트 데이터를 메시지 스트림으로 추상화합니다. 따라서 대기 시간이 짧아지며 여러 데이터 소스 및 분산된 데이터 사용을 더욱 쉽게 지원할 수 있습니다.
- Kafka의 많은 사용자는 여러 스테이지로 구성된 처리 파이프라인에서 원시 입력 데이터가 Kafka 주제에서 소비된 후 추가 소비 또는 후속 처리를 위해 새로운 주제로 집계, 강화 또는 기타 방식으로 변환되는 데이터를 처리합니다. 예를 들어 뉴스 기사를 추천하기 위한 처리 파이프라인은 RSS 피드에서 기사 콘텐츠를 크롤링하여 "기사" 항목에 게시할 수 있습니다. 추가 처리에서는 이 콘텐츠를 정규화하거나 중복 제거하고 정리된 문서 콘텐츠를 새 주제에 게시하며 최종 처리 단계에서 사용자에게 이 콘텐츠를 추천하려고 할 수 있습니다. 이러한 처리 파이프라인은 개별 주제를 기반으로 실시간 데이터 플로우의 그래프를 작성합니다.
- 이벤트 수그리기는 상태 변경이 시간 순서 기록 시퀀스로 기록되는 응용 프로그램 디자인의 스타일입니다. Kafka는 매우 큰 저장 로그 데이터를 지원하므로 이 스타일로 구축된 애플리케이션에 대한 탁월한 백엔드로 활용할 수 있습니다.
- Kafka는 분산 시스템에 대한 일종의 외부 커밋 로그 역할을 할 수 있습니다. 이 로그는 노드 간 데이터를 복제하고 장애가 발생한 노드가 데이터를 복원할 수 있도록 재동기화 메커니즘 역할을 합니다. Kafka의 로그 컴팩션 기능은 이 활용 사례를 지원하는 데 도움이 됩니다.

유창하게

Confluent Platform은 Kafka를 완성하는 엔터프라이즈급 플랫폼으로, 애플리케이션 개발 및 연결 속도를 높이고, 스트림 처리를 통해 혁신을 지원하고, 규모에 따라 엔터프라이즈 운영을 간소화하고, 엄격한 아키텍처 요구 사항을 충족하도록 설계된 고급 기능을 제공합니다. Apache Kafka를 처음 개발한 Confluent는 Kafka 관리 또는 모니터링의 부담을 덜면서 엔터프라이즈급 기능을 통해 Kafka의 이점을 확장해 줍니다. 현재 Fortune 100대 기업 중 80% 이상이 데이터 스트리밍 기술을 사용하고 있으며 대부분 Confluent를 사용하고 있습니다.

왜 **Confluent**인가**?**

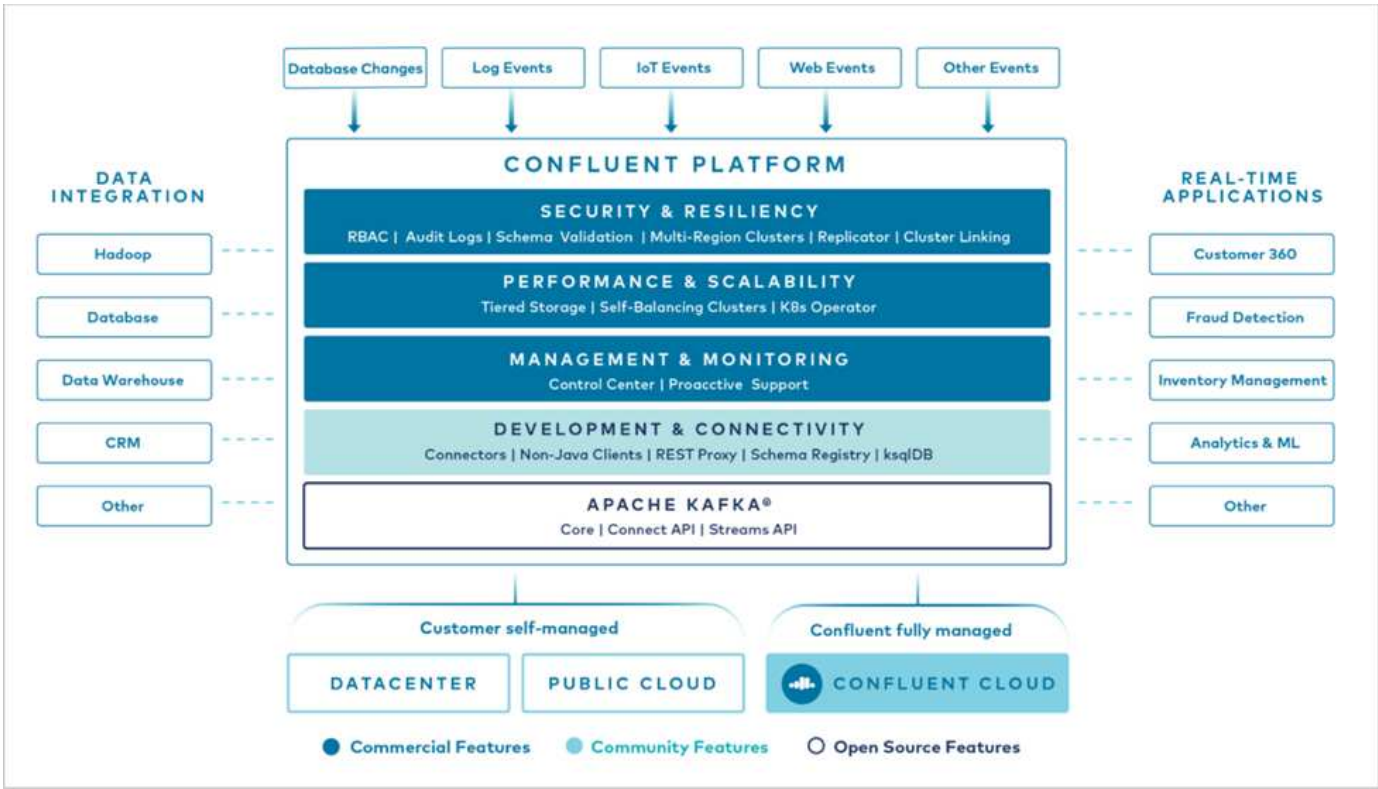
Confluorent는 기록 데이터와 실시간 데이터를 단일 중앙 데이터 소스에 통합하여 완전히 새로운 범주의 최신 이벤트 기반 애플리케이션을 쉽게 구축하고, 범용 데이터 파이프라인을 구축하며, 완전한 확장성, 성능, 안정성으로 강력한 새 사용 사례를 활용할 수 있도록 지원합니다.

Contfluent는 어떤 용도로 사용됩니까**?**

Confluent Platform을 사용하면 데이터가 다른 시스템 간에 어떻게 전송 또는 통합되는지 등의 기본 메커니즘을 걱정하지 않고 데이터에서 비즈니스 가치를 창출하는 방법에 집중할 수 있습니다. 특히 Confluent Platform은 데이터

소스를 Kafka에 연결하고 스트리밍 애플리케이션을 구축하며 Kafka 인프라의 보안, 모니터링 및 관리를 간소화합니다. 현재 Confluent Platform은 금융 서비스, 옴니채널 소매, 자율 자동차, 사기 탐지 등 다양한 산업 전반의 다양한 사용 사례에 사용됩니다. 마이크로서비스, IoT

다음 그림에서는 Confluent Kafka 플랫폼 구성 요소를 보여 줍니다.



Confluent의 이벤트 스트리밍 기술 개요

Confluent Platform의 핵심은 입니다 "아파치 카프카"가장 널리 사용되는 오픈 소스 분산 스트리밍 플랫폼입니다. Kafka의 주요 기능은 다음과 같습니다.

- 레코드 스트림을 게시하고 구독합니다.
- 내결함성이 있는 방식으로 레코드 스트림을 저장합니다.
- 레코드 스트림을 처리합니다.

즉시 사용할 수 있는 Confluorent Platform에는 스키마 레지스트리, REST 프록시, 총 100개 이상의 사전 구축된 Kafka 커넥터 및 ksqlDB도 포함되어 있습니다.

Confluent 플랫폼의 엔터프라이즈 기능 개요

- * Confluent Control Center. * Kafka 관리 및 모니터링을 위한 GUI 기반 시스템. Kafka Connect를 쉽게 관리하고 다른 시스템에 대한 연결을 생성, 편집 및 관리할 수 있습니다.
- Kubernetes를 위한 * Contfluent. * Kubernetes를 위한 Confluent는 Kubernetes 운영자입니다. Kubernetes 운영자는 특정 플랫폼 애플리케이션에 대한 고유한 기능과 요구 사항을 제공하여 Kubernetes의 오케스트레이션 기능을 확장합니다. Confluent Platform의 경우, Kubernetes에서 Kafka의 구축 프로세스를 크게 간소화하고 일반적인 인프라 라이프사이클 작업을 자동화할 수 있습니다.
- * Kafka * 커넥터에 대한 Confluent 커넥터 Kafka Connect API를 사용하여 Kafka를 데이터베이스, 키 값 저장소, 검색 인덱스 및 파일 시스템과 같은 다른 시스템에 연결합니다. Confluorent Hub에는 가장 널리 사용되는 데이터

소스 및 싱크에 대한 다운로드 가능한 커넥터가 있습니다. 여기에는 Confluorent Platform이 포함된 이러한 커넥터의 전체 테스트 및 지원 버전이 포함됩니다. 자세한 내용은 을 참조하십시오 "여기".

- * 자체 밸런싱 클러스터 * 는 자동화된 로드 밸런싱, 장애 감지 및 자동 복구를 제공합니다. 필요에 따라 브로커를 추가하거나 해체할 수 있도록 지원하며 수동 튜닝이 필요하지 않습니다.
- * 연결 클러스터. * 직접 클러스터를 연결하고 링크 브리지를 통해 클러스터 간에 주제를 미러링합니다. 클러스터 링크를 사용하면 멀티 데이터 센터, 멀티 클러스터, 하이브리드 클라우드 구축을 간편하게 설정할 수 있습니다.
- * Confluorent auto data balancer. * 브로커 수, 파티션 크기, 파티션 수 및 클러스터 내의 리더 수에 대한 클러스터를 모니터링합니다. 균형 조정을 통해 트래픽을 재조정함으로써 운영 워크로드에 미치는 영향을 최소화하면서 클러스터 전체에서 짝수 워크로드를 생성할 수 있습니다.
- * Confluent Replicator. * 여러 데이터 센터에서 여러 Kafka 클러스터를 훨씬 쉽게 유지 관리할 수 있습니다.
- * 계층형 스토리지. * 즐겨 사용하는 클라우드 공급자를 사용하여 대량의 Kafka 데이터를 저장할 수 있는 옵션을 제공하므로 운영 부담과 비용이 줄어듭니다. 계층형 스토리지를 사용하면 비용 효율적인 오브젝트 스토리지에 데이터를 보관하고 더 많은 컴퓨팅 리소스가 필요할 때만 브로커를 확장할 수 있습니다.
- * Confluent JMS 클라이언트. * Confluent Platform에는 Kafka용 JMS 호환 클라이언트가 포함되어 있습니다. 이 Kafka 클라이언트는 Kafka 브로커를 백엔드로 사용하여 JMS 1.1 표준 API를 구현합니다. JMS를 사용하는 레거시 애플리케이션이 있고 기존 JMS 메시지 브로커를 Kafka로 교체하려는 경우 유용합니다.
- * Confluent MQTT proxy. * 중간에 MQTT 브로커가 없어도 MQTT 장치 및 게이트웨이에서 Kafka에 직접 데이터를 게시할 수 있는 방법을 제공합니다.
- * Confluent 보안 플러그인 * Confluent 보안 플러그인은 다양한 Confluent 플랫폼 도구 및 제품에 보안 기능을 추가하는 데 사용됩니다. 현재 Confluent REST 프록시에 사용할 수 있는 플러그인이 있어 수신 요청을 인증하고 인증된 보안 주체를 Kafka에 요청에 전파할 수 있습니다. 이렇게 하면 Confluent REST 프록시 클라이언트가 Kafka 브로커의 멀티테넌트 보안 기능을 활용할 수 있습니다.

"다음: Confluent verification"

Confluent 검증

"이전: 기술 개요"

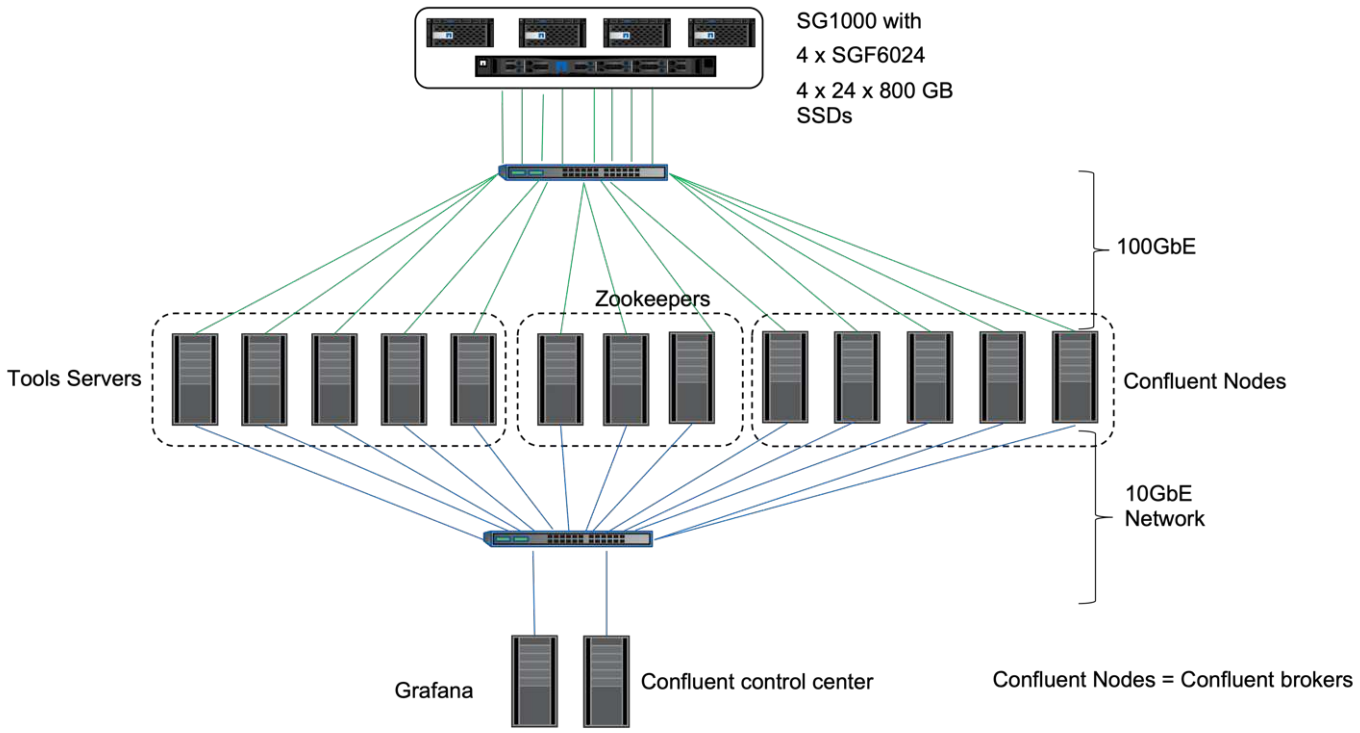
NetApp StorageGRID에서 Confluent Platform 6.2 계층형 스토리지를 사용하여 검증을 수행했습니다. NetApp과 Confluent 팀은 이 검증을 함께 수행하여 검증에 필요한 테스트 사례를 실행했습니다.

Confluent 플랫폼 설정

검증을 위해 다음 설정을 사용했습니다.

검증을 위해 3대의 zookeers, 5개의 브로커, 5개의 테스트 스크립트 실행 서버, 256GB RAM이 장착된 명명된 도구 서버, 16개의 CPU를 사용했습니다. NetApp 스토리지의 경우 SGF6024s 4개로 SG1000 로드 밸런서와 함께 StorageGRID를 사용했습니다. 스토리지와 브로커는 100GbE 연결을 통해 연결되었습니다.

다음 그림은 Confluent 확인에 사용되는 구성의 네트워크 토폴로지를 보여줍니다.



도구 서버는 Confluent 노드에 요청을 보내는 애플리케이션 클라이언트의 역할을 합니다.

Confluent 계층형 스토리지 구성

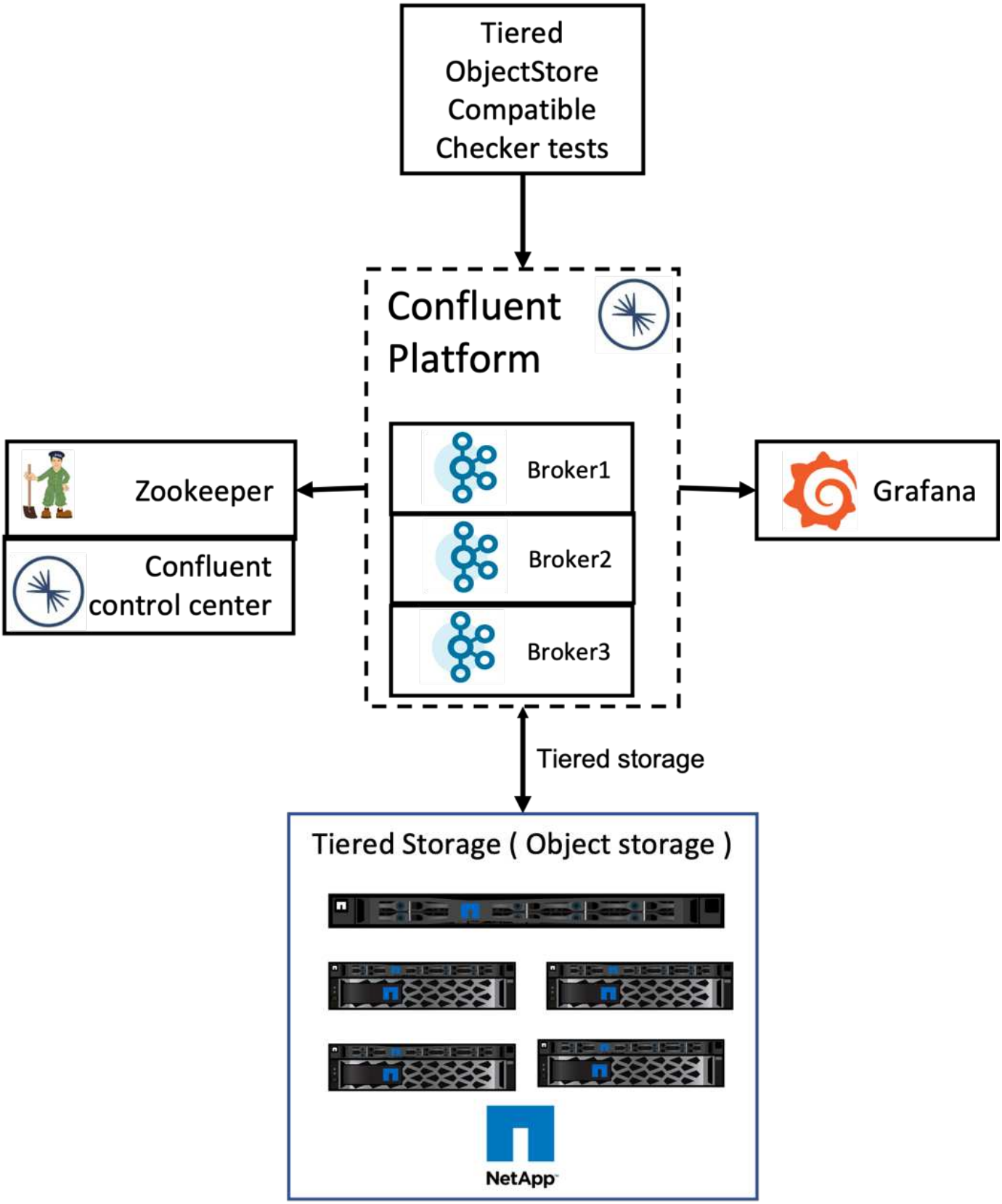
계층화된 스토리지 구성에는 Kafka에서 다음 매개 변수가 필요합니다.

```
Confluent.tier.archiver.num.threads=16
confluent.tier.fetcher.num.threads=32
confluent.tier.enable=true
confluent.tier.feature=true
confluent.tier.backend=S3
confluent.tier.s3.bucket=kafkasgdbucket1-2
confluent.tier.s3.region=us-west-2
confluent.tier.s3.cred.file.path=/data/kafka/.ssh/credentials
confluent.tier.s3.aws.endpoint.override=http://kafkasgd.rtpppe.netapp.com:
10444/
confluent.tier.s3.force.path.style.access=true
```

검증을 위해 StorageGRID를 HTTP 프로토콜과 함께 사용했지만 HTTPS도 작동합니다. 액세스 키와 비밀 키는 confluent.tier.s3.cred.file.path 매개 변수에 제공된 파일 이름에 저장됩니다.

NetApp 오브젝트 스토리지 **- StorageGRID**

StorageGRID에서 정화를 위해 단일 사이트 구성을 구성했습니다.



검증 테스트

검증을 위해 다음 5가지 테스트 사례를 완료했습니다. 이러한 검사는 Trogor 프레임워크에서 실행됩니다. 첫 번째 두 테스트는 기능 테스트이고 나머지 세 가지는 성능 테스트입니다.

객체 저장소 정확도 테스트

이 테스트에서는 오브젝트 저장소 API의 모든 기본 작업(예: get/put/delete)이 계층적 스토리지의 요구에 따라 잘 작동하는지 여부를 확인합니다. 모든 오브젝트 저장소 서비스가 다음 테스트보다 먼저 통과해야 하는 기본 테스트입니다. 통과 또는 실패한 단정적인 테스트입니다.

계층화 기능 정확도 테스트

이 테스트에서는 종단 간 계층형 스토리지 기능이 통과 또는 실패한 적극적 테스트와 잘 작동하는지 여부를 확인합니다. 이 테스트에서는 기본적으로 계층화를 사용하도록 구성되고 핫 세트 크기가 크게 줄어든 테스트 항목을 생성합니다. 새로 만든 테스트 항목으로 이벤트 스트림을 생성하고 브로커가 세그먼트를 개체 저장소에 아카이빙할 때까지 대기한 다음 이벤트 스트림을 사용하여 소비된 스트림이 생성된 스트림과 일치하는지 확인합니다. 이벤트 스트림에 생성되는 메시지 수를 구성할 수 있으므로 테스트 요구에 따라 충분한 크기의 워크로드를 생성할 수 있습니다. 줄어든 핫세트 크기는 소비자가 활성 세그먼트 밖에 있는 페치를 개체 저장소에서만 제공되도록 합니다. 이렇게 하면 읽기에 대한 개체 저장소의 정확성을 테스트하는 데 도움이 됩니다. 객체 저장소 결함 주입 여부와 관계없이 이 테스트를 수행했습니다. StorageGRID의 노드 중 하나에서 서비스 관리자 서비스를 중지하고 엔드 투 엔드 기능이 오브젝트 스토리지에서 작동하는지 확인하여 노드 장애를 시뮬레이션했습니다.

계층 가져오기 벤치마크

이 테스트에서는 계층형 오브젝트 스토리지의 읽기 성능을 검증하고 벤치마크에 의해 생성된 세그먼트에서 과부하된 읽기 요청 범위를 검사했습니다. 이 벤치마크에서 Confluent는 계층 가져오기 요청을 처리하기 위해 사용자 지정 클라이언트를 개발했습니다.

생산 - 워크로드 벤치마크 소비

이 테스트에서는 세그먼트 아카이브를 통해 오브젝트 저장소에서 쓰기 워크로드를 간접적으로 생성했습니다. 소비자 그룹이 세그먼트를 가져올 때 객체 스토리지에서 읽기 워크로드(세그먼트 읽기)가 생성되었습니다. 이 워크로드는 테스트 스크립트에 의해 생성되었습니다. 이 테스트에서는 오브젝트 저장소에서 병렬 스레드의 읽기 및 쓰기 성능을 확인했습니다. 계층화 기능 정확도 테스트에서 보았듯이, 객체 저장소 결함 주입을 사용하여 테스트했으며 포함하지 않았습니다.

보존 워크로드 벤치마크

이 테스트에서는 무거운 주제 보존 워크로드 하에서 오브젝트 저장소의 삭제 성능을 검사했습니다. 보존 워크로드는 테스트 주제와 동시에 많은 메시지를 생성하는 테스트 스크립트를 사용하여 생성되었습니다. 테스트 주제는 공격적인 크기 기반 및 시간 기반 보존 설정으로 구성되었으며, 이로 인해 이벤트 스트림이 개체 저장소에서 지속적으로 제거됩니다. 그런 다음 세그먼트가 아카이브되었습니다. 이로 인해 오브젝트 저장소 삭제 작업의 수행 중개 및 컬렉션에 의해 오브젝트 저장소에서 많은 삭제가 수행되었습니다.

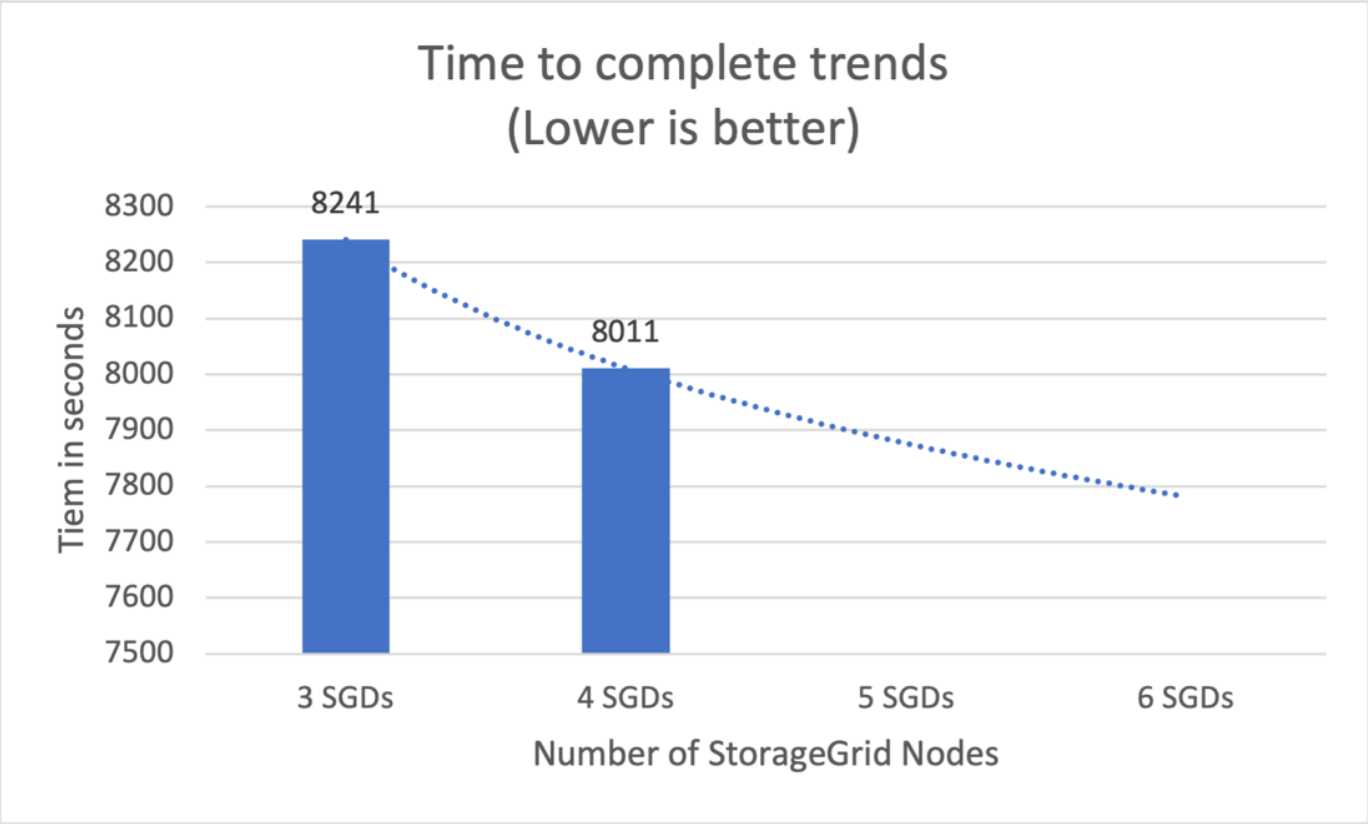
"다음: 확장성의 성능 테스트."

확장성을 갖춘 성능 테스트

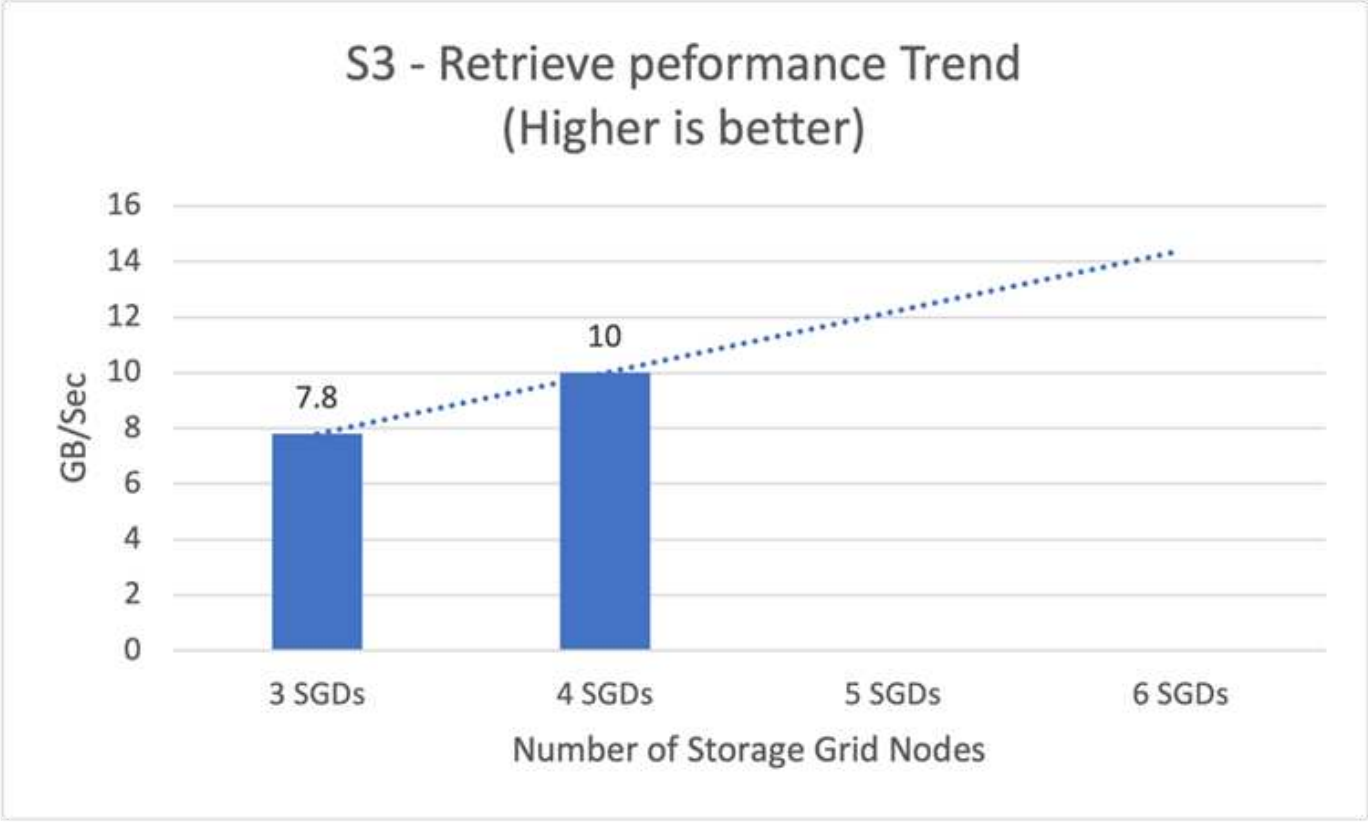
"이전: Confluent verification."

NetApp StorageGRID 설정을 통해 생산자 및 소비자 워크로드를 위한 3~4개의 노드로 계층형 스토리지 테스트를 수행했습니다. 이 테스트에 따르면 완료 시간과 성능 결과는 StorageGRID 노드 수에 정비례합니다. StorageGRID를 설치하려면 최소 3개의 노드가 필요합니다.

- 스토리지 노드의 수가 증가했을 때 생산 및 소비자 작업을 완료하는 데 걸리는 시간이 선형적으로 감소했습니다.



- S3 검색 작업의 성능은 StorageGRID 노드 수에 따라 선형으로 증가합니다. StorageGRID는 최대 200개의 StorgeGRID 노드를 지원합니다.



"다음: Confluent S3 커넥터."

Confluent S3 커넥터

"이전: 확장성의 성능 테스트."

Amazon S3 싱크 커넥터는 Apache Kafka 항목의 데이터를 Avro, JSON 또는 바이트 형식의 S3 오브젝트로 내보냅니다. Amazon S3 싱크 커넥터는 주기적으로 Kafka의 데이터를 폴링하여 S3로 업로드합니다. 분할자는 모든 Kafka 파티션의 데이터를 청크로 분할하는 데 사용됩니다. 각 데이터 청크는 S3 오브젝트로 표시됩니다. 키 이름은 주제, Kafka 파티션 및 이 데이터 청크의 시작 오프셋을 인코딩합니다.

이 설정에서는 Kafka S3 싱크 커넥터를 사용하여 바로 Kafka에서 오브젝트 스토리지의 항목을 읽고 쓰는 방법을 보여줍니다. 이 테스트에서는 독립 실행형 Confluent 클러스터를 사용했지만 이 설정은 분산 클러스터에 적용됩니다.

1. Confluent 웹 사이트에서 Confluent Kafka를 다운로드하십시오.
2. 패키지를 서버의 폴더에 압축을 풉니다.
3. 두 변수를 내보냅니다.

```
Export CONFLUENT_HOME=/data/confluent/confluent-6.2.0
export PATH=$PATH:/data/confluent/confluent-6.2.0/bin
```

4. 독립 실행형 Confluorent Kafka 설정의 경우 클러스터는 "/tmp"에 임시 루트 폴더를 생성합니다. 또한 ZooKeeper, Kafka, 스키마 레지스트리, 연결, ksql-server를 생성합니다. 제어 센터 폴더를 만들고 해당 구성 파일을 '$CONFLUENT_HOME'에서 복사합니다. 다음 예를 참조하십시오.

```
root@stlrx2540m1-108:~# ls -ltr /tmp/confluent.406980/
total 28
drwxr-xr-x 4 root root 4096 Oct 29 19:01 zookeeper
drwxr-xr-x 4 root root 4096 Oct 29 19:37 kafka
drwxr-xr-x 4 root root 4096 Oct 29 19:40 schema-registry
drwxr-xr-x 4 root root 4096 Oct 29 19:45 kafka-rest
drwxr-xr-x 4 root root 4096 Oct 29 19:47 connect
drwxr-xr-x 4 root root 4096 Oct 29 19:48 ksql-server
drwxr-xr-x 4 root root 4096 Oct 29 19:53 control-center
root@stlrx2540m1-108:~#
```

5. ZooKeeper를 구성합니다. 기본 매개 변수를 사용하는 경우 아무것도 변경할 필요가 없습니다.

```
root@stlrx2540m1-108:~# cat
/tmp/confluent.406980/zookeeper/zookeeper.properties  | grep -iv ^#
dataDir=/tmp/confluent.406980/zookeeper/data
clientPort=2181
maxClientCnxns=0
admin.enableServer=false
tickTime=2000
initLimit=5
syncLimit=2
server.179=controlcenter:2888:3888
root@stlrx2540m1-108:~#
```

위 구성에서 서버를 업데이트했습니다. XXX'재산. 기본적으로 Kafka 리더 선택을 위해서는 Zookeepers가 세 개 필요합니다.

6. 고유한 ID로 '/tmp/confluent.406980/zookeeper/data'에 myid 파일을 만들었습니다.

```
root@stlrx2540m1-108:~# cat /tmp/confluent.406980/zookeeper/data/myid
179
root@stlrx2540m1-108:~#
```

myid 파일에 대한 마지막 IP 주소 수를 사용했습니다. Kafka, CONNECT, CONTROL-CENTER, Kafka, Kafka-Rest, ksql-server 및 schema-registry 구성.

7. Kafka 서비스를 시작합니다.

```
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin# confluent
local services  start
The local commands are intended for a single-node development
environment only,
NOT for production usage.

Using CONFLUENT_CURRENT: /tmp/confluent.406980
ZooKeeper is [UP]
Kafka is [UP]
Schema Registry is [UP]
Kafka REST is [UP]
Connect is [UP]
ksqlDB Server is [UP]
Control Center is [UP]
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin#
```

각 구성에 대한 로그 폴더가 있어 문제를 해결하는 데 도움이 됩니다. 경우에 따라 서비스를 시작하는 데 더 많은 시간이 걸릴 수 있습니다. 모든 서비스가 실행 중인지 확인합니다.

8. 'confluent-hub'을 이용하여 Kafka CONNECT를 설치한다.

```
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin# ./confluent-
hub install confluentinc/kafka-connect-s3:latest
The component can be installed in any of the following Confluent
Platform installations:
  1. /data/confluent/confluent-6.2.0 (based on $CONFLUENT_HOME)
  2. /data/confluent/confluent-6.2.0 (where this tool is installed)
Choose one of these to continue the installation (1-2): 1
Do you want to install this into /data/confluent/confluent-
6.2.0/share/confluent-hub-components? (yN) y

Component's license:
Confluent Community License
http://www.confluent.io/confluent-community-license
I agree to the software license agreement (yN) y
Downloading component Kafka Connect S3 10.0.3, provided by Confluent,
Inc. from Confluent Hub and installing into /data/confluent/confluent-
6.2.0/share/confluent-hub-components
Do you want to uninstall existing version 10.0.3? (yN) y
Detected Worker's configs:
  1. Standard: /data/confluent/confluent-6.2.0/etc/kafka/connect-
distributed.properties
  2. Standard: /data/confluent/confluent-6.2.0/etc/kafka/connect-
standalone.properties
  3. Standard: /data/confluent/confluent-6.2.0/etc/schema-
registry/connect-avro-distributed.properties
  4. Standard: /data/confluent/confluent-6.2.0/etc/schema-
registry/connect-avro-standalone.properties
  5. Based on CONFLUENT_CURRENT:
/tmp/confluent.406980/connect/connect.properties
  6. Used by Connect process with PID 15904:
/tmp/confluent.406980/connect/connect.properties
Do you want to update all detected configs? (yN) y
Adding installation directory to plugin path in the following files:
  /data/confluent/confluent-6.2.0/etc/kafka/connect-
distributed.properties
  /data/confluent/confluent-6.2.0/etc/kafka/connect-
standalone.properties
  /data/confluent/confluent-6.2.0/etc/schema-registry/connect-avro-
distributed.properties
  /data/confluent/confluent-6.2.0/etc/schema-registry/connect-avro-
standalone.properties
  /tmp/confluent.406980/connect/connect.properties
  /tmp/confluent.406980/connect/connect.properties
```

```
Completed
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin#
```

confluent-hub install confluentinc/Kafka-connect-S3:10.0.3'을 사용하여 특정 버전을 설치할 수도 있습니다.

9. 기본적으로 '/data/confluent/confluent-6.2.0/share/confluent-hub-components/confluententinc-kafka-connect-s3'에 confluentinc-kafka-connect-s3이 설치됩니다.
10. 새로운 'confluentinc-kafka-connect-s3'으로 플러그인 경로를 업데이트합니다.

```
root@stlrx2540m1-108:~# cat /data/confluent/confluent-
6.2.0/etc/kafka/connect-distributed.properties | grep plugin.path
#
plugin.path=/usr/local/share/java,/usr/local/share/kafka/plugins,/opt/co
nnectors,
plugin.path=/usr/share/java,/data/zookeeper/confluent/confluent-
6.2.0/share/confluent-hub-components,/data/confluent/confluent-
6.2.0/share/confluent-hub-components,/data/confluent/confluent-
6.2.0/share/confluent-hub-components/confluentinc-kafka-connect-s3
root@stlrx2540m1-108:~#
```

11. Confluent 서비스를 중지하고 다시 시작합니다.

```
confluent local services  stop
confluent local services  start
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin# confluent
local services  status
The local commands are intended for a single-node development
environment only,
NOT for production usage.

Using CONFLUENT_CURRENT: /tmp/confluent.406980
Connect is [UP]
Control Center is [UP]
Kafka is [UP]
Kafka REST is [UP]
ksqlDB Server is [UP]
Schema Registry is [UP]
ZooKeeper is [UP]
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin#
```

12. '/root/.aws/credentials' 파일에서 액세스 ID와 비밀 키를 설정한다.

```
root@stlrx2540m1-108:~# cat /root/.aws/credentials
[default]
aws_access_key_id = xxxxxxxxxxxx
aws_secret_access_key = xxxxxxxxxxxxxxxxxxxxxxxxx
root@stlrx2540m1-108:~#
```

13. 버킷에 도달할 수 있는지 확인합니다.

```
root@stlrx2540m4-01:~# aws s3 –endpoint-url
http://kafkasgd.rtpppe.netapp.com:10444 ls kafkasgdbucket1-2
2021-10-29 21:04:18       1388 1
2021-10-29 21:04:20       1388 2
2021-10-29 21:04:22       1388 3
root@stlrx2540m4-01:~#
```

14. S3 및 버킷 구성에 대해 S3-싱크 속성 파일을 구성합니다.

```
root@stlrx2540m1-108:~# cat /data/confluent/confluent-
6.2.0/share/confluent-hub-components/confluentinc-kafka-connect-
s3/etc/quickstart-s3.properties | grep -v ^#
name=s3-sink
connector.class=io.confluent.connect.s3.S3SinkConnector
tasks.max=1
topics=s3_testtopic
s3.region=us-west-2
s3.bucket.name=kafkasgdbucket1-2
store.url=http://kafkasgd.rtpppe.netapp.com:10444/
s3.part.size=5242880
flush.size=3
storage.class=io.confluent.connect.s3.storage.S3Storage
format.class=io.confluent.connect.s3.format.avro.AvroFormat
partitioner.class=io.confluent.connect.storage.partitioner.DefaultPartit
ioner
schema.compatibility=NONE
root@stlrx2540m1-108:~#
```

15. S3 버킷으로 몇 개의 레코드를 가져옵니다.

```
kafka-avro-console-producer --broker-list localhost:9092 --topic
s3_topic \
--property
value.schema='{"type":"record","name":"myrecord","fields":[{"name":"f1",
"type":"string"}]}'
{"f1": "value1"}
{"f1": "value2"}
{"f1": "value3"}
{"f1": "value4"}
{"f1": "value5"}
{"f1": "value6"}
{"f1": "value7"}
{"f1": "value8"}
{"f1": "value9"}
```

16. S3 싱크 커넥터를 로드합니다.

```
root@stlrx2540m1-108:~# confluent local services connect connector load
s3-sink  --config /data/confluent/confluent-6.2.0/share/confluent-hub-
components/confluentinc-kafka-connect-s3/etc/quickstart-s3.properties
The local commands are intended for a single-node development
environment only,
NOT for production usage.
https://docs.confluent.io/current/cli/index.html
{
  "name": "s3-sink",
  "config": {
    "connector.class": "io.confluent.connect.s3.S3SinkConnector",
    "flush.size": "3",
    "format.class": "io.confluent.connect.s3.format.avro.AvroFormat",
    "partitioner.class":
"io.confluent.connect.storage.partitioner.DefaultPartitioner",
    "s3.bucket.name": "kafkasgdbucket1-2",
    "s3.part.size": "5242880",
    "s3.region": "us-west-2",
    "schema.compatibility": "NONE",
    "storage.class": "io.confluent.connect.s3.storage.S3Storage",
    "store.url": "http://kafkasgd.rtpppe.netapp.com:10444/",
    "tasks.max": "1",
    "topics": "s3_testtopic",
    "name": "s3-sink"
  },
  "tasks": [],
  "type": "sink"
}
root@stlrx2540m1-108:~#
```

17. S3 싱크 상태를 확인합니다.

```
root@stlrx2540m1-108:~# confluent local services connect connector
status s3-sink
The local commands are intended for a single-node development
environment only,
NOT for production usage.
https://docs.confluent.io/current/cli/index.html
{
  "name": "s3-sink",
  "connector": {
    "state": "RUNNING",
    "worker_id": "10.63.150.185:8083"
  },
  "tasks": [
    {
      "id": 0,
      "state": "RUNNING",
      "worker_id": "10.63.150.185:8083"
    }
  ],
  "type": "sink"
}
root@stlrx2540m1-108:~#
```

18. 로그를 확인하여 S3 싱크가 항목을 수락할 준비가 되었는지 확인합니다.

```
root@stlrx2540m1-108:~# confluent local services connect log
```

19. Kafka의 주제를 확인하십시오.

```
kafka-topics --list --bootstrap-server localhost:9092
…
connect-configs
connect-offsets
connect-statuses
default_ksql_processing_log
s3_testtopic
s3_topic
s3_topic_new
root@stlrx2540m1-108:~#
```

20. S3 버킷의 오브젝트를 확인합니다.

```
root@stlrx2540m1-108:~# aws s3 --endpoint-url
http://kafkasgd.rtpppe.netapp.com:10444 ls --recursive kafkasgdbucket1-
2/topics/
2021-10-29 21:24:00        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000000.avro
2021-10-29 21:24:00        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000003.avro
2021-10-29 21:24:00        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000006.avro
2021-10-29 21:24:08        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000009.avro
2021-10-29 21:24:08        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000012.avro
2021-10-29 21:24:09        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000015.avro
root@stlrx2540m1-108:~#
```

21. 내용을 확인하려면 다음 명령을 실행하여 각 파일을 S3에서 로컬 파일 시스템으로 복사합니다.

```
root@stlrx2540m1-108:~# aws s3 --endpoint-url
http://kafkasgd.rtpppe.netapp.com:10444 cp s3://kafkasgdbucket1-
2/topics/s3_testtopic/partition=0/s3_testtopic+0+0000000000.avro
tes.avro
download: s3://kafkasgdbucket1-
2/topics/s3_testtopic/partition=0/s3_testtopic+0+0000000000.avro to
./tes.avro
root@stlrx2540m1-108:~#
```

22. 레코드를 인쇄하려면 avro-tools-1.11.0.1.jar (에서 사용 가능)를 사용합니다 "아파치 아카이브")를 클릭합니다.

```
root@stlrx2540m1-108:~# java -jar /usr/src/avro-tools-1.11.0.1.jar
tojson tes.avro
21/10/30 00:20:24 WARN util.NativeCodeLoader: Unable to load native-
hadoop library for your platform... using builtin-java classes where
applicable
{"f1":"value1"}
{"f1":"value2"}
{"f1":"value3"}
root@stlrx2540m1-108:~#
```

"다음: ConfFluent 자체 재조정 클러스터."

Confluent Self-Balancing 클러스터

"이전: Kafka S3 커넥터."

이전에 Kafka 클러스터를 관리했다면 수동으로 파티션을 다른 브로커에 재할당하여 클러스터 전체에서 워크로드가 균형 있게 조정되도록 하는 데 따르는 문제에 익숙할 것입니다. Kafka가 대규모로 구축된 조직의 경우 대량의 데이터를 개각하는 일은 까다롭고 번거로우며 위험할 수 있습니다. 특히 클러스터 위에 미션 크리티컬 애플리케이션을 구축하는 경우에는 더욱 그렇습니다. 그러나 가장 작은 Kafka 사용 사례에서도 이 프로세스는 시간이 많이 걸리며 오류가 발생하기 쉽습니다.

이 실습에서는 Confluent Self-Balancing 클러스터 기능을 테스트하여 클러스터 토폴로지 변경 또는 불균등한 로드를 기반으로 재조정을 자동화했습니다. Confluent rebalance 테스트는 노드 장애 또는 확장 노드에서 브로커 간의 데이터 재조정이 필요한 경우 새 브로커를 추가하는 시간을 측정하는 데 도움이 됩니다. 기존 Kafka 구성에서는 클러스터의 증가에 따라 재조정할 데이터의 양이 증가하지만 계층형 스토리지에서는 소량의 데이터로 제한됩니다. NetApp의 검증을 바탕으로, 계층화된 스토리지에서 재조정은 기존 Kafka 아키텍처에서 몇 초 또는 몇 분 만에 이루어지게 되며, 클러스터의 확장에 따라 선형으로 증가합니다.

자체 밸런싱 클러스터에서는 파티션 재조정이 완전히 자동화되어 Kafka의 처리량을 최적화하고 브로커 확장을 가속화하며 대규모 클러스터를 실행하는 데 따른 운영 부담을 줄일 수 있습니다. 안정적인 상태에서 자체 균형 조정 클러스터는 브로커 전체의 데이터 불균형을 모니터링하고 지속적으로 파티션을 재할당하여 클러스터 성능을 최적화합니다. 플랫폼을 확장하거나 축소할 때 자체 균형 조정 클러스터는 새로운 브로커의 존재 또는 이전 브로커의 제거를 자동으로 인식하고 후속 파티션 재할당을 트리거합니다. 이를 통해 브로커를 쉽게 추가하고 해체할 수 있으므로 Kafka 클러스터의 유연성이 근본적으로 높아집니다. 이러한 이점은 수동 개입, 복잡한 수학 또는 재할당을 분할하는 데 수반되는 인적 오류의 위험 없이 제공됩니다. 따라서 데이터 재조정이 훨씬 더 빠르게 완료되고, 클러스터를 지속적으로 감독해야 하는 대신 더 가치 있는 이벤트 스트리밍 프로젝트에 집중할 수 있습니다.

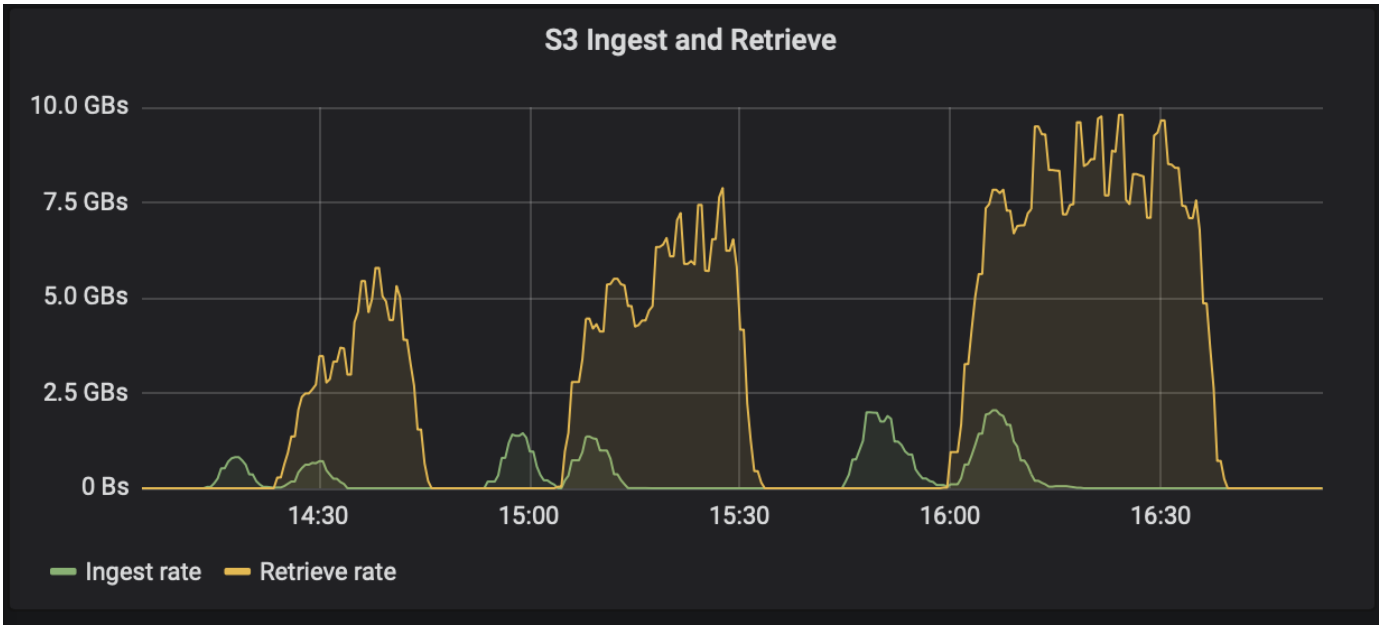
"다음은 모범 사례 지침입니다."

모범 사례 지침

"이전: Confluent 자체 재조정 클러스터"

- NetApp의 검증을 기반으로 S3 오브젝트 스토리지는 데이터를 유창하게 유지하는 데 가장 적합합니다.
- 처리량이 높은 SAN(특히 FC)을 사용하여 브로커의 핫 데이터 또는 로컬 디스크를 유지할 수 있습니다. 왜냐하면, ContFluent 계층형 스토리지 구성에서 브로커 데이터 디렉토리에 있는 데이터의 크기는 데이터가 오브젝트 스토리지로 이동되는 세그먼트 크기 및 보존 시간을 기준으로 합니다.
- 오브젝트 저장소는 세그먼트일 때 더 나은 성능을 제공합니다. 바이트 수는 더 높고 512MB를 테스트했습니다.
- Kafka에서는 해당 주제에 대해 생성된 각 레코드에 대한 키 또는 값(바이트)의 길이가 length.key.value 매개 변수에 의해 제어됩니다. StorageGRID의 경우 S3 오브젝트 수집 및 검색 성능이 더 높은 값으로 향상되었습니다. 예를 들어, 512바이트는 5.8GBps 검색, 1024바이트는 7.5GBps S3 검색, 2048바이트는 10Gbps 가까이 제공

다음 그림은 length.key.value를 기준으로 S3 오브젝트 수집 및 조회 결과를 나타낸 것이다.



- * Kafka 튜닝. * 계층형 스토리지의 성능을 향상시키려면 TierFetcherNumThreads 및 TierArchivernumThreads를 늘릴 수 있습니다. 일반적으로 TierFetchernumThreads를 물리적 CPU 코어 수와 일치하도록 늘리고 TierArchivernumThreads를 CPU 코어 수의 절반으로 늘리고자 합니다. 예를 들어, 서버 속성에서 8개의 물리적 코어가 있는 컴퓨터를 사용하는 경우 confluent.tier.fetcher.num.threads=8 및 confluent.tier.Archiver.num.threads=4를 설정합니다.
- * 항목 삭제에 대한 시간 간격 * 주제가 삭제되면 객체 저장소에서 로그 세그먼트 파일 삭제가 즉시 시작되지 않습니다. 대신 이 파일을 삭제하기 전에 기본값이 3시간인 시간 간격이 있습니다. confluent.tier.topic.delete.check.interval.ms 구성을 수정하여 이 간격의 값을 변경할 수 있습니다. 주제 또는 클러스터를 삭제하는 경우 해당 버킷의 오브젝트도 수동으로 삭제할 수 있습니다.
- * 계층형 스토리지 내부 항목에 대한 ACL. * 온-프레미스 배포의 권장 모범 사례는 계층형 스토리지에 사용되는 내부 항목에 대해 ACL 허가자를 활성화하는 것입니다. 이 데이터에 대한 액세스를 브로커 사용자에게만 제한하려면 ACL 규칙을 설정합니다. 이렇게 하면 내부 항목이 안전하게 보호되며 계층형 스토리지 데이터 및 메타데이터에 대한 무단 액세스가 방지됩니다.

```
kafka-acls --bootstrap-server localhost:9092 --command-config adminclient-
configs.conf \
--add --allow-principal User:<kafka> --operation All --topic "_confluent-
tier-state"
```

ⓘ 사용자 '<Kafka>'를 배포의 실제 브로커 교장으로 바꿉니다.

예를 들어, "confluent-tier-state" 명령은 계층형 스토리지의 내부 항목에 대한 ACL을 설정합니다. 현재 계층형 스토리지와 관련된 내부 주제는 하나뿐입니다. 이 예제에서는 내부 항목의 모든 작업에 대해 Kafka의 주요 권한을 제공하는 ACL을 만듭니다.

"다음: 사이징."

사이징

"이전: 모범 사례 지침."

Kafka 사이징은 단순, 세분화, 역방향 및 파티션의 4가지 구성 모드로 수행할 수 있습니다.

단순함

단순 모드는 최초 Apache Kafka 사용자 또는 초기 상태 사용 사례에 적합합니다. 이 모드에서는 처리량 MBps, 읽기 팬아웃, 보존 및 리소스 사용률(기본값 60%)과 같은 요구 사항을 제공합니다. 또한 온프레미스(베어 메탈, VMware, Kubernetes 또는 OpenStack) 또는 클라우드와 같은 환경에 진입할 수 있습니다. 이 정보를 바탕으로 Kafka 클러스터의 크기를 조정하면 브로커, zookeeper, Apache Kafka 연결 작업자, 스키마 레지스트리, REST 프록시, ksqlDB 및 Confluorent 제어 센터에 필요한 서버 수가 제공됩니다.

계층형 스토리지의 경우 Kafka 클러스터의 크기를 조정할 수 있는 세분화된 구성 모드를 고려해 보십시오. 세밀한 모드는 숙련된 Apache Kafka 사용자나 잘 정의된 사용 사례에 적합합니다. 이 섹션에서는 생산자, 스트림 프로세서 및 소비자를 위한 크기를 조정하는 방법에 대해 설명합니다.

프로듀서

Apache Kafka의 생산자(예: 네이티브 클라이언트, REST 프록시 또는 Kafka 커넥터)를 설명하기 위해 다음 정보를 제공합니다.

- * 이름. * Spark.
- * Producer type. * 응용 프로그램 또는 서비스, 프록시(RDBMS, MQTT, 기타) 및 기존 데이터베이스(RDBMS, NoSQL, 기타). "모름"을 선택할 수도 있습니다.
- * 초당 이벤트 수 평균 처리량 * (예: 1,000,000).
- * 최대 처리량 * 초당 이벤트 수(예: 4,000,000).
- * 평균 메시지 크기. * (바이트), 압축되지 않음(예: 최대 1MB, 1000).
- * 메시지 형식. * 옵션은 Avro, JSON, 프로토콜 버퍼, 바이너리, 텍스트, “잘 모르겠군요.” 및 기타.
- * 복제 계수. * 옵션은 1, 2, 3(Confluent recommendation), 4, 5, 또는 6.
- * 보존 시간. * 1일(예:) 데이터를 Apache Kafka에 얼마나 오랫동안 저장하기를 원하십니까? 무한 시간 동안 임의의 단위로 -1을 입력합니다. 이 계산기는 보존 기간이 10년으로 무한 경우를 가정합니다.
- "Enable Tiered Storage to Decrease Broker Count and Allow for Infinite Storage?" 확인란을 선택합니다.
- 계층형 스토리지를 사용하는 경우 보존 필드는 브로커에 로컬로 저장된 데이터의 핫 세트를 제어합니다. 아카이브 보존 필드는 아카이브 오브젝트 스토리지에 데이터가 저장되는 기간을 제어합니다.
- * 아카이브 스토리지 보존. * 1년(예:) 아카이브 스토리지에 데이터를 얼마나 오랫동안 저장하려는 경우 무한 기간 동안 임의의 단위로 -1을 입력합니다. 이 계산기는 무한 보존을 위해 10년을 유지하는 것으로 가정합니다.
- * Growth Multiplier. * 1(예:) 이 매개 변수의 값이 현재 처리량을 기반으로 하는 경우 1로 설정합니다. 추가 성장에 따라 크기를 조정하려면 이 매개 변수를 성장 배수로 설정합니다.
- * 생산자 인스턴스 수. * 10(예:) 얼마나 많은 프로듀서 인스턴스가 실행됩니까? 이 입력은 CPU 부하를 사이징 계산에 통합하기 위해 필요합니다. 빈 값은 CPU 로드가 계산에 포함되지 않았음을 나타냅니다.

이 예제 입력에 따라 크기를 조정하면 생산자에 다음과 같은 영향을 줍니다.

- 압축되지 않은 바이트의 평균 처리량: 1Gbps. 압축되지 않은 바이트 단위의 최대 처리량: 4Gbps. 압축된 바이트의 평균 처리량: 400Mbps 압축된 바이트 단위의 최대 처리량: 1.6GBps. 이 값은 기본 60% 압축률을 기반으로 합니다(이 값은 변경할 수 있음).
  - 필요한 총 온브로커 핫 세트 스토리지 수: 31,104TB(복제 포함), 압축. 총 비브로커 아카이브 스토리지 필요: 378,432TB, 압축. 사용 "https://fusion.netapp.com" StorageGRID 사이징:

스트림 프로세서는 아파치 Kafka로부터 데이터를 소비하고 아파치 Kafka로 다시 생산하는 애플리케이션 또는 서비스를 설명해야 합니다. 대부분의 경우 ksqlDB 또는 Kafka 스트림에 빌드됩니다.

- * 이름. * Spark streamer.
- * 처리 시간. * 이 프로세서는 단일 메시지를 처리하는 데 얼마나 걸립니까?
  - 1ms(단순한 상태 비저장 변환) [예], 10ms(Stateful 인메모리 작업).
  - 100ms(stateful 네트워크 또는 디스크 작동), 1000ms(타사 REST 호출)
  - 이 매개 변수를 벤치마킹하고 시간이 얼마나 걸리는지 정확히 알고 있습니다.
- * 출력 보존. * 1일(예) 스트림 프로세서는 Apache Kafka로 출력을 다시 생성합니다. 이 출력 데이터를 Apache Kafka에 얼마나 오랫동안 저장하기를 원하십니까? 무한 기간 동안 임의의 단위로 -1을 입력합니다.
- "계층 스토리지를 사용하여 브로커 수를 줄이고 무한 확장 스토리지를 허용하시겠습니까?" 확인란을 선택합니다.
- * 아카이브 스토리지 보존. * 1년(예:) 아카이브 스토리지에 데이터를 얼마나 오랫동안 저장하려는 경우 무한 기간 동안 임의의 단위로 -1을 입력합니다. 이 계산기는 무한 보존을 위해 10년을 유지하는 것으로 가정합니다.
- * 출력 통과 백분율. * 100(예:) 스트림 프로세서는 Apache Kafka로 출력을 다시 생성합니다. Apache Kafka로 다시 출력되는 인바운드 처리량의 비율은 얼마입니까? 예를 들어, 인바운드 처리량이 20Mbps이고 이 값이 10이면 출력 처리량은 2Mbps입니다.
- 어떤 응용 프로그램에서 이 정보를 읽습니까? 프로듀서 유형 기반 사이징에 사용되는 이름인 “Spark”를 선택합니다. 위의 입력을 기반으로 스트림 프로세스 인스턴스 및 주제 파티션 예상에 대한 사이징의 영향을 예상할 수 있습니다.
- 이 스트림 프로세서 응용 프로그램에는 다음과 같은 수의 인스턴스가 필요합니다. 들어오는 항목에는 이러한 많은 파티션이 필요할 수 있습니다. 이 매개 변수를 확인하려면 Confluent에 문의하십시오.
  - 성장 승수가 없는 평균 처리량에 대해 1,000개
  - 최대 처리량에 대해 4,000(성장 승수 없음)
  - 성장 승수가 포함된 평균 처리량의 경우 1,000개
  - 최대 처리량에 대해 4,000(성장 승수 포함)

소비자

Apache Kafka의 데이터를 사용하고 Apache Kafka로 다시 생성하지 않는 애플리케이션 또는 서비스(예: 네이티브 클라이언트 또는 Kafka Connector)에 대해 설명하십시오.

- * 이름. * Spark 소비자.
- * 처리 시간. * 이 소비자는 단일 메시지를 처리하는 데 얼마나 걸립니까?
  - 1ms(예: 로깅 같은 간단하고 상태 비저장 작업)
  - 10ms(데이터 저장소에 대한 빠른 쓰기)
  - 100ms(데이터 저장소에 대한 느린 쓰기)
  - 1000ms(타사 휴면 통화)
  - 알려진 기간의 다른 벤치마크 프로세스
- * 소비자 유형. * 기존 데이터 저장소(RDBMS, NoSQL, 기타)에 대한 애플리케이션, 프록시 또는 싱크
- 어떤 응용 프로그램에서 이 정보를 읽습니까? 이 매개 변수를 이전에 결정된 생산자 및 스트림 사이징에 연결합니다.

위의 입력 내용에 따라 소비자 인스턴스 및 주제 파티션 추정치에 대한 사이징을 결정해야 합니다. 소비자 응용

프로그램에는 다음과 같은 수의 인스턴스가 필요합니다.

- 평균 처리량에 대해 2,000개, 성장 승수 없음
- 최대 처리량에 대해 8,000개, 성장 승수 없음
- 성장 승수를 포함한 평균 처리량에 대해 2,000개
- 성장 승수를 포함한 최대 처리량에 대해 8,000개

들어오는 주제에는 이 수의 파티션도 필요할 것입니다. 확인하려면 Confluent에 문의하십시오.

생산자, 스트림 프로세서 및 소비자에 대한 요구 사항 외에도 다음과 같은 추가 요구 사항을 제공해야 합니다.

- * 재생성 시간. * 예: 4시간. Apache Kafka 브로커 호스트에 장애가 발생하고 데이터가 손실되며 장애가 발생한 호스트를 대체하기 위해 새 호스트를 프로비저닝하는 경우 이 새 호스트 재구축 속도는 얼마나 빨라야 합니까? 값을 알 수 없는 경우 이 매개 변수를 비워 둡니다.
- * 리소스 활용률 목표(백분율) * 예: 60. 평균 처리량 중에 호스트를 얼마나 활용하기를 원하십니까? Confluent는 Confluent 셀프 밸런싱 클러스터를 사용하고 있지 않는 한 60%의 사용률을 권장합니다. 이 경우 활용률이 더 높을 수 있습니다.

환경에 대해 설명하십시오

- * 클러스터가 어떤 환경에서 실행됩니까? * Amazon Web Services, Microsoft Azure, Google 클라우드 플랫폼, 베어 메탈 온프레미스, VMware 온프레미스, 사내에 OpenStack 또는 온프레미스에 Kubernates가 있습니까?
- * 호스트 세부 정보. * 코어 수: 48(예:), 네트워크 카드 유형(10GbE, 40GbE, 16GbE, 1GbE 또는 다른 유형).
- * 스토리지 볼륨. * 호스트: 12(예:) 호스트당 지원되는 하드 드라이브 또는 SSD 수는 몇 개입니까? Confluent는 호스트당 12개의 하드 드라이브를 권장합니다.
- * 스토리지 용량/볼륨(GB). * 1000(예:) 단일 볼륨에서 몇 기가바이트의 스토리지를 저장할 수 있습니까? Confluent에서는 1TB 디스크를 권장합니다.
- * 스토리지 구성. * 스토리지 볼륨은 어떻게 구성됩니까? Confluent는 Raid10에서 모든 Confluent 기능을 이용할 것을 권장합니다. JBOD, SAN, RAID 1, RAID 0, RAID 5, 및 기타 유형도 지원됩니다.
- * 단일 볼륨 처리량(Mbps). * 125(예:) 단일 스토리지 볼륨이 초당 메가바이트 단위로 읽거나 쓸 수 있는 속도는 얼마나 됩니까? Confluent는 일반적으로 125MBps 처리량의 표준 하드 드라이브를 권장합니다.
- * 메모리 용량(GB). * 64(예:)

환경 변수를 결정한 후 클러스터 크기를 선택합니다. 위에 표시된 예시 매개 변수를 토대로 Confluent Kafka에 대한 다음 사이징을 결정했습니다.

- * 아파치 Kafka. * 브로커 수: 22. 클러스터가 스토리지에 바인딩되어 있습니다. 호스트 수를 줄이고 무한 스토리지를 허용하도록 계층형 스토리지를 설정하는 것이 좋습니다.
- * Apache ZooKeeper. * Count:5; Apache Kafka Connect 작업자: Count:2; Schema Registry: Count:2; REST Proxy: Count:2; ksqlDB:Count:2; Confluorent Control Center: Count:1.

사용 사례를 염두에 두고 플랫폼 팀에 리버스 모드를 사용합니다. 파티션 모드를 사용하여 단일 항목에 필요한 파티션 수를 계산합니다. 을 참조하십시오 https://eventsizer.io 역 및 파티션 모드에 따른 크기 조정.

"다음: 결론."

결론

"이전: 사이징."

이 문서에서는 검증 테스트, 계층형 스토리지 성능 결과, 튜닝, Confluent S3 커넥터 및 셀프 밸런싱 기능을 포함하여 NetApp 스토리지와 함께 Contuent Tiered Storage를 사용하기 위한 모범 사례 지침을 제공합니다. ILM 정책, 검증을 위한 다양한 성능 테스트 및 산업 표준 S3 API를 통한 유창한 성능을 고려할 때 NetApp StorageGRID 오브젝트 스토리지는 유창한 계층형 스토리지를 위한 최적의 선택입니다.

추가 정보를 찾을 수 있는 위치

이 문서에 설명된 정보에 대해 자세히 알아보려면 다음 문서 및/또는 웹 사이트를 검토하십시오.

- 아파치 카프카란

  "https://www.confluent.io/what-is-apache-kafka/"

- NetApp 제품 설명서

  "https://www.netapp.com/support-and-training/documentation/"

- S3 싱크 매개 변수 세부 정보

  "https://docs.confluent.io/kafka-connect-s3-sink/current/configuration_options.html#s3-configuration-options"

- 아파치 카프카

  "https://en.wikipedia.org/wiki/Apache_Kafka"

- Confluent 플랫폼의 무한 스토리지

  "https://www.confluent.io/blog/infinite-kafka-storage-in-confluent-platform/"

- ContFluent Tiered Storage - 모범 사례 및 사이징

  "https://docs.confluent.io/platform/current/kafka/tiered-storage.html#best-practices-and-recommendations"

- Confluent 플랫폼용 Amazon S3 싱크 커넥터

  "https://docs.confluent.io/kafka-connect-s3-sink/current/overview.html"

- Kafka 사이징

  "https://eventsizer.io"

- StorageGRID 사이징

  "https://fusion.netapp.com/"

- Kafka 활용 사례

  "https://kafka.apache.org/uses"

- confluent platform 6.0의 자체 균형 조정 Kafka 클러스터

  "https://www.confluent.io/blog/self-balancing-kafka-clusters-in-confluent-platform-6-0/"

  "https://www.confluent.io/blog/confluent-platform-6-0-delivers-the-most-powerful-event-streaming-platform-to-date/"

버전 기록

버전	날짜	문서 버전 기록
버전 1.0	2021년 12월	최초 릴리스.

NetApp 하이브리드 클라우드 데이터 솔루션 - 고객 사용 사례를 기반으로 Spark 및 Hadoop

TR-4657: NetApp 하이브리드 클라우드 데이터 솔루션 - 고객 사용 사례를 기반으로 Spark 및 Hadoop

NetApp의 Karthikeyan Nagalingam 및 Sathish Thyagarajan

본 문서는 NetApp AFF 및 FAS 스토리지 시스템, NetApp Cloud Volumes ONTAP, NetApp 연결형 스토리지, Spark 및 Hadoop용 NetApp FlexClone 기술을 사용하는 하이브리드 클라우드 데이터 솔루션에 대해 설명합니다. 이러한 솔루션 아키텍처를 통해 고객은 자신의 환경에 적합한 데이터 보호 솔루션을 선택할 수 있습니다. NetApp은 고객과의 상호 작용 및 비즈니스 사용 사례를 기반으로 이러한 솔루션을 설계했습니다. 이 문서에서는 다음과 같은 자세한 정보를 제공합니다.

- Spark 및 Hadoop 환경 및 고객의 당면 과제를 해결하기 위해 데이터 보호가 필요한 이유
- NetApp 비전 및 구성 요소와 서비스를 기반으로 하는 Data Fabric
- 이러한 구성 요소를 사용하여 유연한 데이터 보호 워크플로우를 구축하는 방법
- 실제 고객 사용 사례를 기반으로 한 여러 아키텍처의 장단점을 설명합니다. 각 활용 사례는 다음과 같은 구성 요소를 제공합니다.
  - 고객 시나리오
  - 요구사항 및 당면 과제
  - 해결하세요
  - 솔루션 요약

Hadoop 데이터 보호를 선택해야 하는 이유

Hadoop 및 Spark 환경에서는 다음과 같은 문제를 해결해야 합니다.

- * 소프트웨어 또는 사용자 오류. * Hadoop 데이터 작업을 수행하는 동안 소프트웨어 업데이트에 사람의 실수가 발생할 수 있으며, 이로 인해 작업에 예상치 못한 결과가 발생할 수 있습니다. 이 경우 오류나 부당한 결과를 방지하려면 데이터를 보호해야 합니다. 예를 들어, 소프트웨어 업데이트가 제대로 실행되지 않아 트래픽 신호 데이터를 일반 텍스트 형식으로 제대로 분석하지 못하는 새로운 기능이 추가되었습니다. 이 소프트웨어는 JSON 및 기타 비 텍스트 파일 형식을 분석하여 실시간 트래픽 제어 분석 시스템을 통해 데이터 포인트가 누락된 예측 결과를 생성합니다. 이 상황은 출력 결함을 초래하여 교통 신호에서 사고를 일으킬 수 있습니다. 데이터 보호는 이전 작업

중인 응용 프로그램 버전으로 빠르게 롤백할 수 있는 기능을 제공하여 이 문제를 해결할 수 있습니다.

- * 규모와 확장성 * 데이터 소스와 볼륨의 수가 계속 증가함에 따라 분석 데이터의 크기가 매일 증가하고 있습니다. 소셜 미디어, 모바일 앱, 데이터 분석 및 클라우드 컴퓨팅 플랫폼은 현재 빅데이터 시장의 주요 데이터 소스로서 빠르게 증가하고 있으며, 따라서 정확한 데이터 운영을 위해 데이터를 보호해야 합니다.
- * Hadoop의 기본 데이터 보호. * Hadoop에는 데이터를 보호하는 기본 명령이 있지만 이 명령은 백업 중에 데이터의 일관성을 제공하지 않습니다. 디렉토리 레벨 백업만 지원합니다. Hadoop에서 생성된 스냅샷은 읽기 전용이며 백업 데이터를 직접 재사용하는 데 사용할 수 없습니다.

Hadoop 및 **Spark** 고객의 데이터 보호 당면 과제

Hadoop 및 Spark 고객의 일반적인 과제는 데이터 보호 중에 운영 클러스터의 성능에 부정적인 영향을 주지 않고 백업 시간을 단축하고 백업 안정성을 높이는 것입니다.

또한 고객은 RPO(복구 시점 목표) 및 RTO(복구 시간 목표) 다운타임을 최소화하고 사내 및 클라우드 기반 재해 복구 사이트를 제어하여 비즈니스 연속성을 최적화해야 합니다. 이 제어 기능은 일반적으로 엔터프라이즈급 관리 툴을 사용하는 데서 비롯됩니다.

Hadoop과 Spark 환경은 데이터 볼륨이 엄청나며 증가하기 때문에 복잡합니다. 하지만 데이터가 도착하는 속도는 점점 증가하고 있습니다. 이 시나리오를 통해 소스 데이터에서 효율적인 최신 DevTest 및 QA 환경을 빠르게 생성하기가 어렵습니다. NetApp은 이러한 과제를 인식하고 이 백서에 제공된 솔루션을 제공합니다.

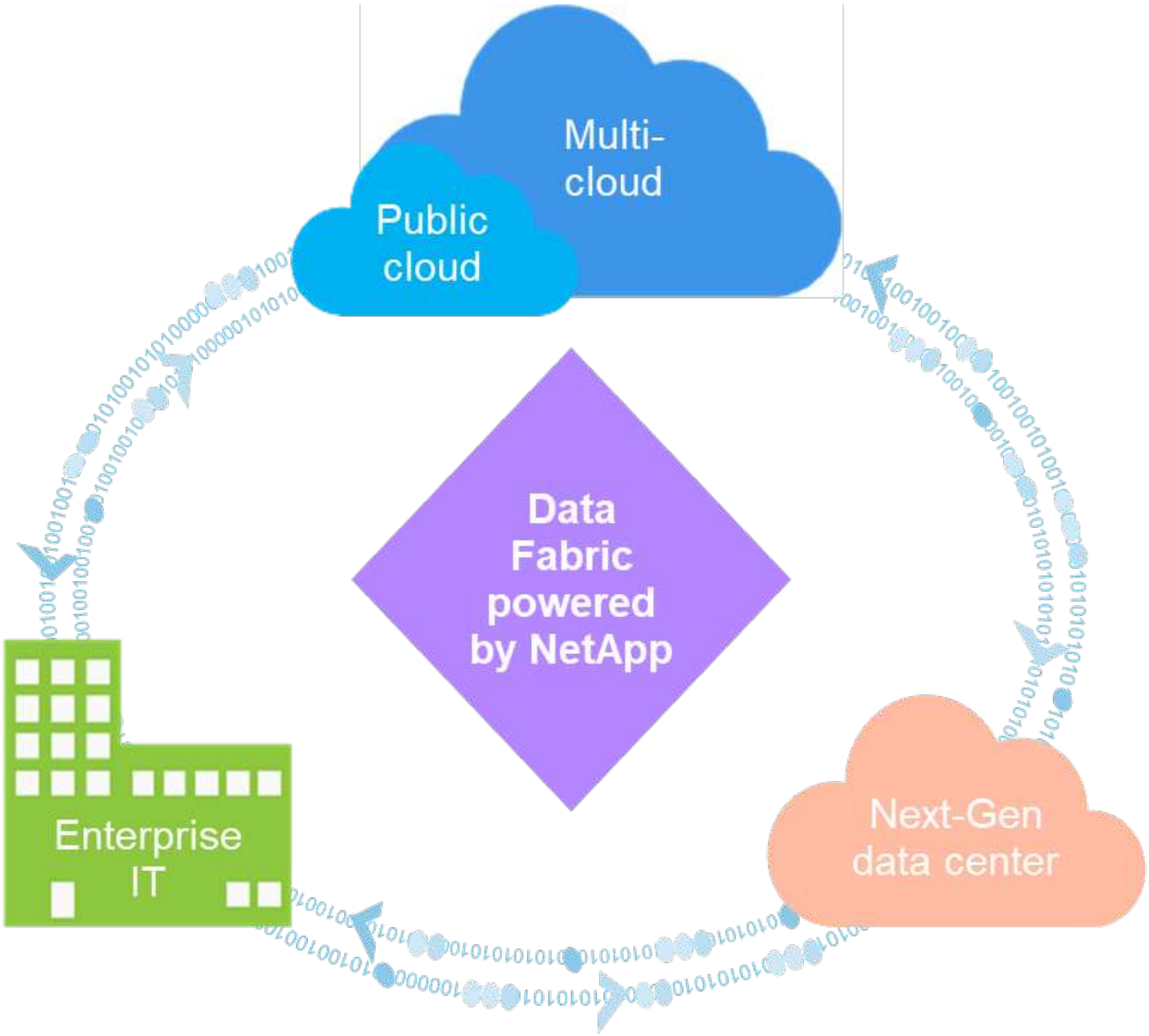
"다음 단계로: NetApp에서 제공하는 빅데이터 아키텍처 관련 Data Fabric"

NetApp에서 제공하는 빅데이터 아키텍처 관련 **Data Fabric**

"이전: 솔루션 개요"

NetApp이 제공하는 Data Fabric은 클라우드 및 사내 환경에서 데이터 관리를 단순화하고 통합하여 디지털 전환을 가속합니다.

NetApp이 제공하는 Data Fabric은 데이터 가시성과 통찰력, 데이터 액세스 및 제어, 데이터 보호 및 보안을 위한 일관성 있는 통합 데이터 관리 서비스 및 애플리케이션(구성 요소)을 제공합니다(아래 그림 참조).



검증된 **Data Fabric** 고객 사용 사례

NetApp이 제공하는 Data Fabric은 고객에게 다음과 같은 9가지 검증된 사용 사례를 제공합니다.

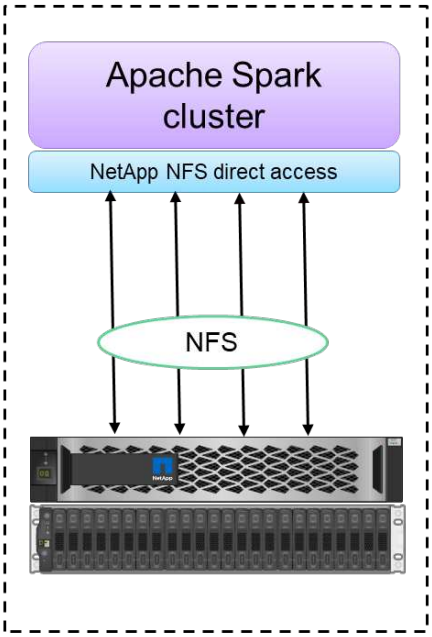
- 분석 워크로드 가속화
- DevOps 혁신 가속
- 클라우드 호스팅 인프라 구축
- 클라우드 데이터 서비스 통합
- 데이터 보호 및 보안
- 비정형데이터의 최적화
- 데이터 센터의 효율성 확보
- 데이터 통찰력 및 제어 제공
- 단순화 및 자동화

본 문서는 9가지 사용 사례 중 2가지(솔루션 포함)를 다룹니다.

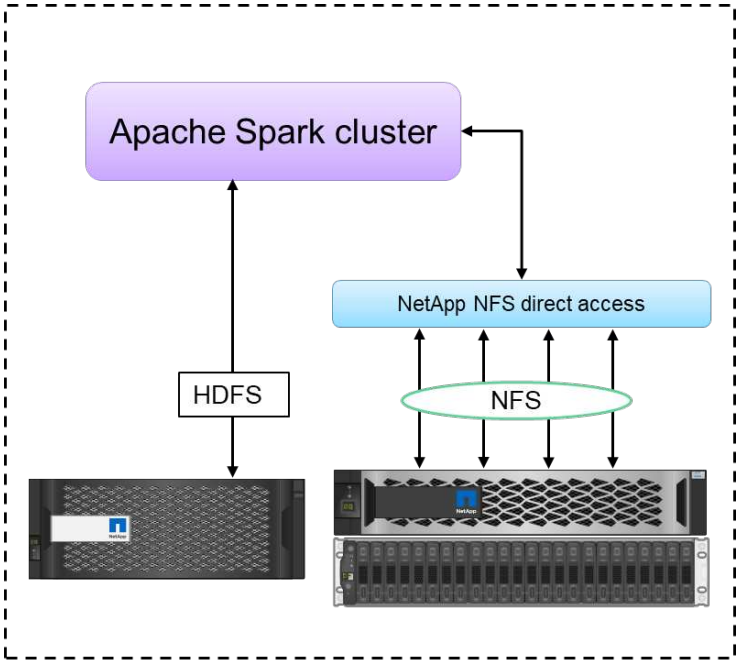
- 분석 워크로드 가속화
- 데이터 보호 및 보안

NetApp NFS 직접 액세스

아래 그림에 표시된 NetApp NFS 직접 액세스(이전의 NetApp In-Place Analytics Module)를 사용하면 데이터를 이동하거나 복사하지 않고도 기존 또는 새로운 NFSv3 또는 NFSv4 데이터에 대해 빅데이터 분석 작업을 실행할 수 있습니다. 여러 데이터 복제본을 방지하고 소스와 데이터를 동기화할 필요가 없습니다. 예를 들어 금융 부문에서 데이터를 한 위치에서 다른 위치로 이동하는 것은 쉬운 일이 아닌 법적 의무를 준수해야 합니다. 이 시나리오에서는 NetApp NFS 직접 액세스가 원래의 위치에서 재무 데이터를 분석합니다. 또 다른 주요 이점은 NetApp NFS 직접 액세스를 사용하여 기본 Hadoop 명령을 사용하여 Hadoop 데이터를 간편하게 보호하고 NetApp의 강력한 데이터 관리 포트폴리오를 활용하여 데이터 보호 워크플로우를 활성화할 수 있다는 것입니다.



Configuration 1: NFS as primary storage



Configuration 2: HDFS and NFS in single Spark cluster

NetApp NFS 직접 액세스에서는 Hadoop/Spark 클러스터에 다음과 같은 두 가지 유형의 구축 옵션을 제공합니다.

- 기본적으로 Hadoop/Spark 클러스터는 데이터 스토리지와 기본 파일 시스템에 HDFS(Hadoop Distributed File System)를 사용합니다. NetApp NFS 직접 액세스는 기본 HDFS를 NFS 스토리지로 대체하여 NFS 데이터에 대한 직접 분석 작업을 지원합니다.
- 다른 구축 옵션에서 NetApp NFS 직접 액세스는 NFS를 단일 Hadoop/Spark 클러스터의 HDFS와 함께 추가 스토리지로 구성할 수 있도록 지원합니다. 이 경우 고객은 NFS 내보내기를 통해 데이터를 공유하고 HDFS 데이터와 함께 동일한 클러스터에서 데이터를 액세스할 수 있습니다.

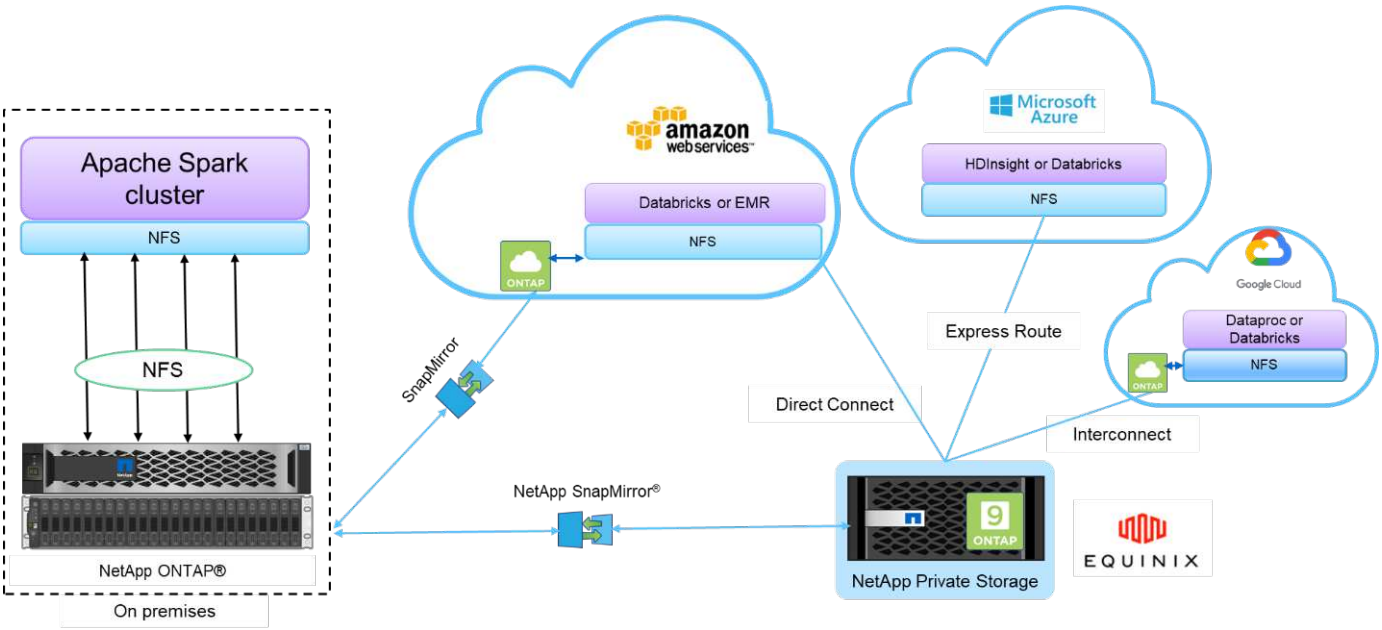
NetApp NFS 직접 액세스를 사용할 때의 주요 이점은 다음과 같습니다.

- 현재 위치의 데이터를 분석하여 분석 데이터를 HDFS와 같은 Hadoop 인프라스트럭처로 이동하는 데 시간과 성능이 많이 소모되는 작업을 방지합니다.
- 복제본 수를 3개부터 1개로 줄입니다.

- 사용자가 컴퓨팅 및 스토리지를 분리하여 독립적으로 확장할 수 있습니다.
- ONTAP의 강력한 데이터 관리 기능을 활용하여 엔터프라이즈 데이터 보호 기능을 제공합니다.
- Hortonworks 데이터 플랫폼에서 인증되었습니다.
- 하이브리드 데이터 분석을 구축할 수 있습니다.
- 동적 다중 스레드 기능을 활용하여 백업 시간을 단축합니다.

빅 데이터를 위한 구성 요소

NetApp 기반의 Data Fabric은 아래 그림과 같이 데이터 액세스, 제어, 보호 및 보안을 위한 데이터 관리 서비스와 애플리케이션(구성 요소)을 통합합니다.



위 그림의 구성 요소는 다음과 같습니다.

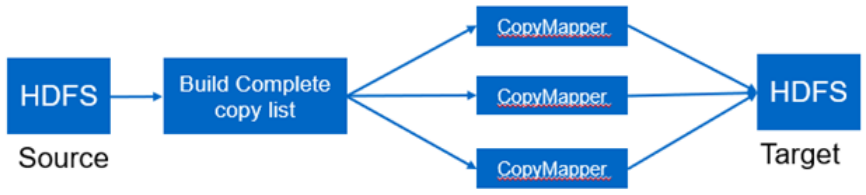
- * NetApp NFS 직접 액세스 * 는 추가 소프트웨어 또는 드라이버 요구사항 없이 최신 Hadoop 및 Spark 클러스터를 NetApp NFS 볼륨에 직접 액세스할 수 있도록 지원합니다.
- * NetApp Cloud Volumes ONTAP 및 클라우드 볼륨 서비스. * Microsoft Azure 클라우드 서비스의 AWS(Amazon Web Services) 또는 ANF(Azure NetApp Files)에서 실행되는 ONTAP를 기반으로 하는 소프트웨어 정의 연결형 스토리지.
- NetApp SnapMirror 기술 *. 사내 및 ONTAP 클라우드 또는 NPS 인스턴스 간에 데이터 보호 기능을 제공합니다.
- * 클라우드 서비스 공급자 * 이러한 공급자에는 AWS, Microsoft Azure, Google Cloud 및 IBM Cloud가 포함됩니다.
- * PaaS. * AWS의 EMR(Amazon Elastic MapReduce) 및 Databricks와 같은 클라우드 기반 분석 서비스와 Microsoft Azure HDInsight 및 Azure Databricks

"이제 Hadoop 데이터 보호와 NetApp이 모두 필요합니다."

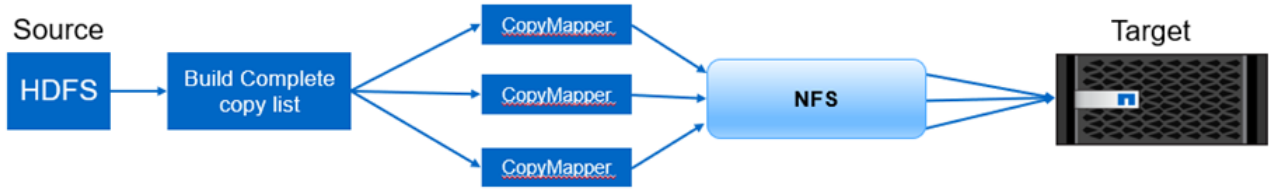
Hadoop 데이터 보호 및 NetApp

"이전: NetApp의 빅데이터 아키텍처 기반 Data Fabric"

Hadoop DistCp는 대규모 클러스터 간 및 클러스터 간 복제에 사용되는 기본 툴입니다. 아래 그림에 표시된 Hadoop DistCp 기본 프로세스는 MapReduce와 같은 Hadoop 기본 툴을 사용하여 HDFS 소스에서 해당 타겟으로 Hadoop 데이터를 복제하는 일반적인 백업 워크플로우입니다. NetApp NFS 직접 액세스를 통해 고객은 NFS를 Hadoop DistCp 툴의 타겟 타겟으로 설정하여 MapReduce를 통해 HDFS 소스에서 NFS 공유로 데이터를 복사할 수 있습니다. NetApp NFS 직접 액세스는 DistCp 툴을 위한 NFS 드라이버 역할을 합니다.



Hadoop Distcp Basic Process



Hadoop Distcp and NetApp

"다음: Hadoop 데이터 보호 활용 사례 개요"

Hadoop 데이터 보호 활용 사례 개요

"이전: Hadoop 데이터 보호 및 NetApp"

이 섹션에서는 이 백서의 초점을 구성하는 데이터 보호 활용 사례에 대해 자세히 설명합니다. 나머지 섹션에서는 고객 문제(시나리오), 요구 사항 및 당면 과제, 솔루션 등 각 사용 사례에 대한 자세한 정보를 제공합니다.

사용 사례 1: Hadoop 데이터 백업

이 활용 사례에서 In-Place Analytics Module은 대형 금융 기관에서 긴 백업 시간을 24시간 이상으로 단축하면서 몇 시간 미만으로 단축하도록 지원했습니다.

사용 사례 2: 클라우드에서 사내로 백업 및 재해 복구

NetApp에서 제공하는 Data Fabric을 구성 요소로 사용함으로써 대규모 브로드캐스트 회사는 주문형, 즉각적인 데이터 전송 같은 다양한 데이터 전송 모드에 따라 클라우드 데이터를 사내 데이터 센터에 백업하는 요구사항을 충족시킬 수 있었습니다. 또는 Hadoop/Spark 클러스터 로드를 기반으로 합니다.

사용 사례 3: 기존 Hadoop 데이터에 대해 DevTest 활성화

NetApp 솔루션을 통해 온라인 음악 유통업체는 다양한 지점에서 공간 효율적인 다중 Hadoop 클러스터를 신속하게 구축하여 보고서를 생성하고 예약된 정책을 사용하여 일일 DevTest 작업을 실행할 수 있었습니다.

사용 사례 4: 데이터 보호 및 멀티 클라우드 연결

한 대형 서비스 공급자는 NetApp이 제공하는 Data Fabric을 사용하여 다양한 클라우드 인스턴스에서 고객에게 멀티

클라우드 분석을 제공합니다.

사용 사례 **5:** 분석 워크로드 가속화

가장 큰 금융 서비스 및 투자 은행 중 하나는 NetApp 네트워크 연결 스토리지 솔루션을 사용하여 I/O 대기 시간을 줄이고 정량 재무 분석 플랫폼을 가속화했습니다.

"다음: 활용 사례 1 - Hadoop 데이터 백업"

사용 사례 **1: Hadoop** 데이터 백업

"이전: Hadoop 데이터 보호 활용 사례 개요"

시나리오

이 시나리오에서는 고객이 대규모 사내 Hadoop 저장소를 보유하고 있으며 재해 복구를 위해 백업하려고 합니다. 그러나 고객의 현재 백업 솔루션은 비용이 많이 들고 24시간 이상의 긴 백업 윈도우에 어려움을 겪고 있습니다.

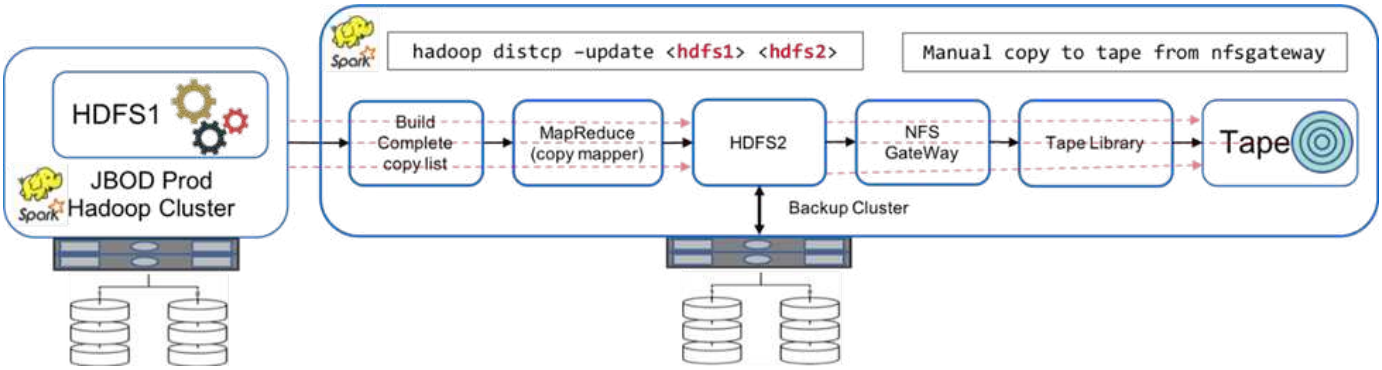
요구사항 및 당면 과제

이 사용 사례의 주요 요구사항과 과제는 다음과 같습니다.

- 소프트웨어 하위 호환성:
  - 제안된 대체 백업 솔루션은 운영 Hadoop 클러스터에 사용되는 현재 실행 중인 소프트웨어 버전과 호환되어야 합니다.
- 확정된 SLA를 충족하기 위해 제안된 대체 솔루션은 매우 낮은 RPO 및 RTO를 달성해야 합니다.
- NetApp 백업 솔루션을 통해 생성된 백업은 데이터 센터에서 로컬로 구축된 Hadoop 클러스터뿐만 아니라 원격 사이트의 재해 복구 위치에서 실행되는 Hadoop 클러스터에서도 사용할 수 있습니다.
- 제안된 솔루션은 비용 효율적이어야 합니다.
- 제안 솔루션은 백업 시간 동안 현재 실행 중인 운영 중인 분석 작업의 성능 영향을 줄여야 합니다.

고객의 기존 백업 솔루션

아래 그림은 원래 Hadoop 네이티브 백업 솔루션을 보여 줍니다.



운영 데이터는 중간 백업 클러스터를 통해 테이프로 보호됩니다.

- HDFS1 데이터는 'Hadoop distcp-update<hdfs1><hdfs2>' 명령을 실행하여 HDFS2에 복사됩니다.

- 백업 클러스터는 NFS 게이트웨이 역할을 하며 테이프 라이브러리를 통해 Linux 'CP' 명령을 통해 테이프에 데이터를 수동으로 복사합니다.

원래 Hadoop 네이티브 백업 솔루션의 이점은 다음과 같습니다.

- 이 솔루션은 Hadoop 기본 명령을 기반으로 하므로 사용자가 새로운 절차를 배울 필요가 없습니다.
- 이 솔루션은 업계 표준 아키텍처와 하드웨어를 활용합니다.

원래 Hadoop 네이티브 백업 솔루션의 단점은 다음과 같습니다.

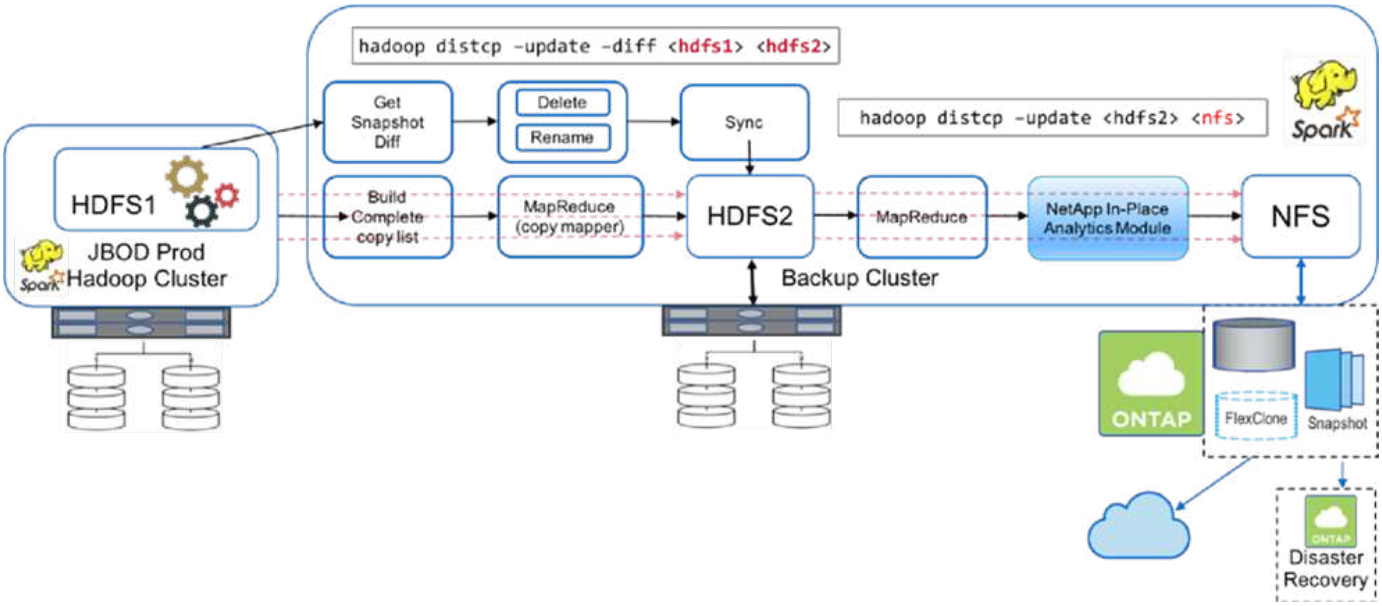
- 긴 백업 시간이 24시간을 초과하므로 운영 데이터가 취약해집니다.
- 백업 시간 동안 클러스터 성능이 크게 저하되었습니다.
- 테이프에 복사하는 것은 수동 프로세스입니다.
- 백업 솔루션은 필요한 하드웨어 및 수동 프로세스에 필요한 인력 시간의 측면에서 비용이 많이 듭니다.

백업 솔루션

이러한 당면 과제와 요구 사항을 바탕으로 기존 백업 시스템을 고려하여 세 가지 가능한 백업 솔루션을 제안하였습니다. 다음 하위 섹션에서는 이러한 세 가지 백업 솔루션 각각에 대해 설명합니다. 솔루션 A에서 솔루션 C까지의 레이블이 지정되어 있습니다

솔루션 **A**

솔루션 A는 아래 그림과 같이 데이터 이동 없는 분석 모듈을 백업 Hadoop 클러스터에 추가하여 NetApp NFS 스토리지 시스템에 대한 보조 백업을 허용함으로써 테이프 요구 사항을 제거합니다.



솔루션 A에 대한 자세한 작업은 다음과 같습니다.

- 운영 Hadoop 클러스터에는 보호가 필요한 HDFS에 고객의 분석 데이터가 있습니다.
- HDFS가 포함된 백업 Hadoop 클러스터는 데이터의 중간 위치로 작동합니다. JBOD(Just a Bunch of Disks)는 운영 및 백업 Hadoop 클러스터 모두에서 HDFS용 스토리지를 제공합니다.

- Hadoop distcp –update –diff <hdfs1><hdfs2>' 명령을 실행하여 운영 클러스터 HDFS에서 백업 클러스터 HDFS로 Hadoop 운영 데이터를 보호합니다.

> Hadoop 스냅샷은 운영 환경에서 백업 Hadoop 클러스터로 데이터를 보호하는 데 사용됩니다.

- NetApp ONTAP 스토리지 컨트롤러는 NFS로 내보낸 볼륨을 제공하며 백업 Hadoop 클러스터에 프로비저닝됩니다.
- MapReduce와 여러 개의 mapper를 활용하는 "Hadoop distcp" 명령을 실행하면 데이터 이동 없는 분석 모듈을 사용하여 분석 데이터가 백업 Hadoop 클러스터에서 NFS로 보호됩니다.

  NetApp 스토리지 시스템의 NFS에 데이터가 저장된 후 NetApp Snapshot, SnapRestore 및 FlexClone 기술을 사용하여 필요에 따라 Hadoop 데이터를 백업, 복원, 복제할 수 있습니다.

> SnapMirror 기술을 사용하여 Hadoop 데이터를 클라우드뿐 아니라 재해 복구 위치에도 보호할 수 있습니다.

솔루션 A의 이점은 다음과 같습니다.

- Hadoop 운영 데이터는 백업 클러스터로부터 보호됩니다.
- HDFS 데이터는 NFS를 통해 보호되므로 클라우드 및 재해 복구 위치에 대한 보호가 가능합니다.
- 백업 작업을 백업 클러스터로 오프로드하여 성능을 향상시킵니다.
- 수동 테이프 작업이 필요 없습니다
- NetApp 툴을 통해 엔터프라이즈 관리 기능을 지원합니다.
- 기존 환경을 최소한으로 변경해야 합니다.
- 비용 효율적인 솔루션입니다.

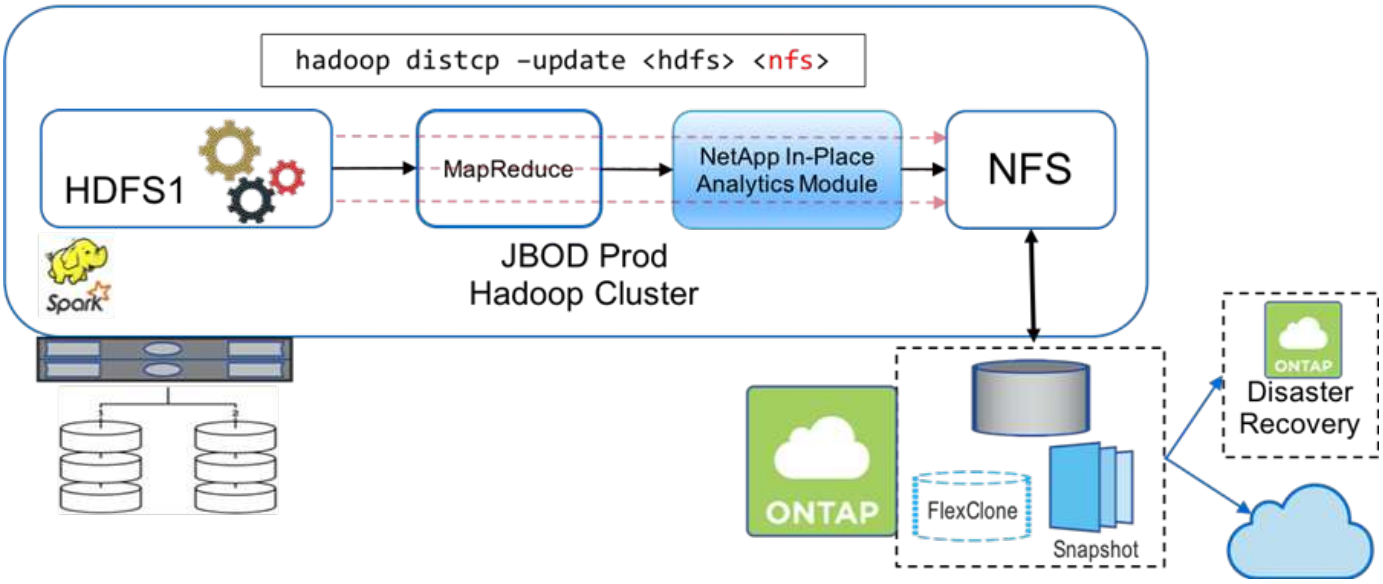
이 솔루션의 단점은 성능을 향상시키기 위해 백업 클러스터와 추가 매퍼가 필요하다는 것입니다.

이 고객은 단순성, 비용, 전반적인 성능 때문에 최근에 솔루션 A를 배포했습니다.

이 솔루션에서는 JBOD 대신 ONTAP의 SAN 디스크를 사용할 수 있습니다. 이 옵션은 백업 클러스터 스토리지 로드를 ONTAP로 오프로드하지만, 단점은 SAN 패브릭 스위치가 필요하다는 점입니다.

해결 방법 **B**

솔루션 B는 운영 Hadoop 클러스터에 데이터 이동 없는 분석 모듈을 추가하여 아래 그림과 같이 백업 Hadoop 클러스터가 필요하지 않습니다.



솔루션 B에 대한 자세한 작업은 다음과 같습니다.

- NetApp ONTAP 스토리지 컨트롤러는 운영 Hadoop 클러스터에 NFS 내보내기를 프로비저닝합니다.

  Hadoop 네이티브 'Hadoop distcp' 명령은 Hadoop 데이터를 데이터 이동 없는 분석 모듈을 통해 운영 클러스터 HDFS에서 NFS로 보호합니다.

- NetApp 스토리지 시스템의 NFS에 데이터가 저장된 후에는 Snapshot, SnapRestore 및 FlexClone 기술을 사용하여 필요에 따라 Hadoop 데이터를 백업, 복원, 복제할 수 있습니다.

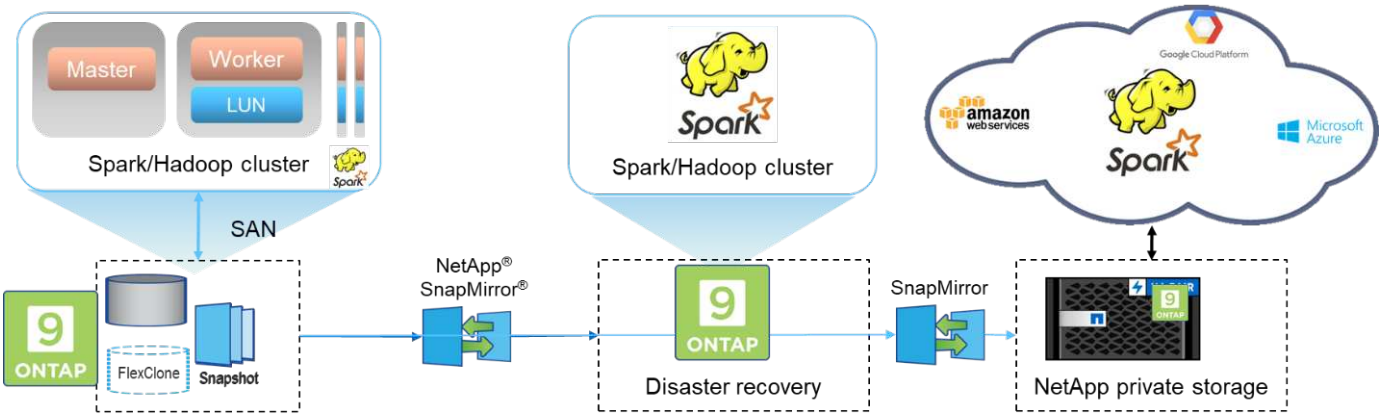
솔루션 B의 이점은 다음과 같습니다.

- 운영 클러스터는 백업 솔루션에 맞게 약간 수정되어 구축이 간소화되고 추가 인프라스트럭처 비용이 절감됩니다.
- 백업 작업을 위한 백업 클러스터는 필요하지 않습니다.
- HDFS 운영 데이터는 NFS 데이터 변환 시 보호됩니다.
- 이 솔루션을 사용하면 NetApp 툴을 통해 엔터프라이즈 관리 기능을 수행할 수 있습니다.

이 솔루션의 단점은 프로덕션 클러스터에 구현되어 운영 클러스터에 추가 관리자 작업을 추가할 수 있다는 것입니다.

솔루션 C

솔루션 C에서는 아래 그림과 같이 NetApp SAN 볼륨을 HDFS 스토리지용 Hadoop 운영 클러스터에 직접 프로비저닝합니다.



솔루션 C에 대한 자세한 단계는 다음과 같습니다.

- NetApp ONTAP SAN 스토리지는 HDFS 데이터 스토리지를 위한 운영 Hadoop 클러스터에서 프로비저닝됩니다.
- NetApp Snapshot 및 SnapMirror 기술은 운영 Hadoop 클러스터의 HDFS 데이터를 백업하는 데 사용됩니다.
- 백업이 스토리지 계층에 있기 때문에 스냅샷 복사본 백업 프로세스 중에 Hadoop/Spark 클러스터의 운영에 미치는 성능 영향은 없습니다.

> ⓘ 스냅샷 기술은 데이터 크기에 관계없이 몇 초 내에 백업을 완료합니다.

솔루션 C의 이점은 다음과 같습니다.

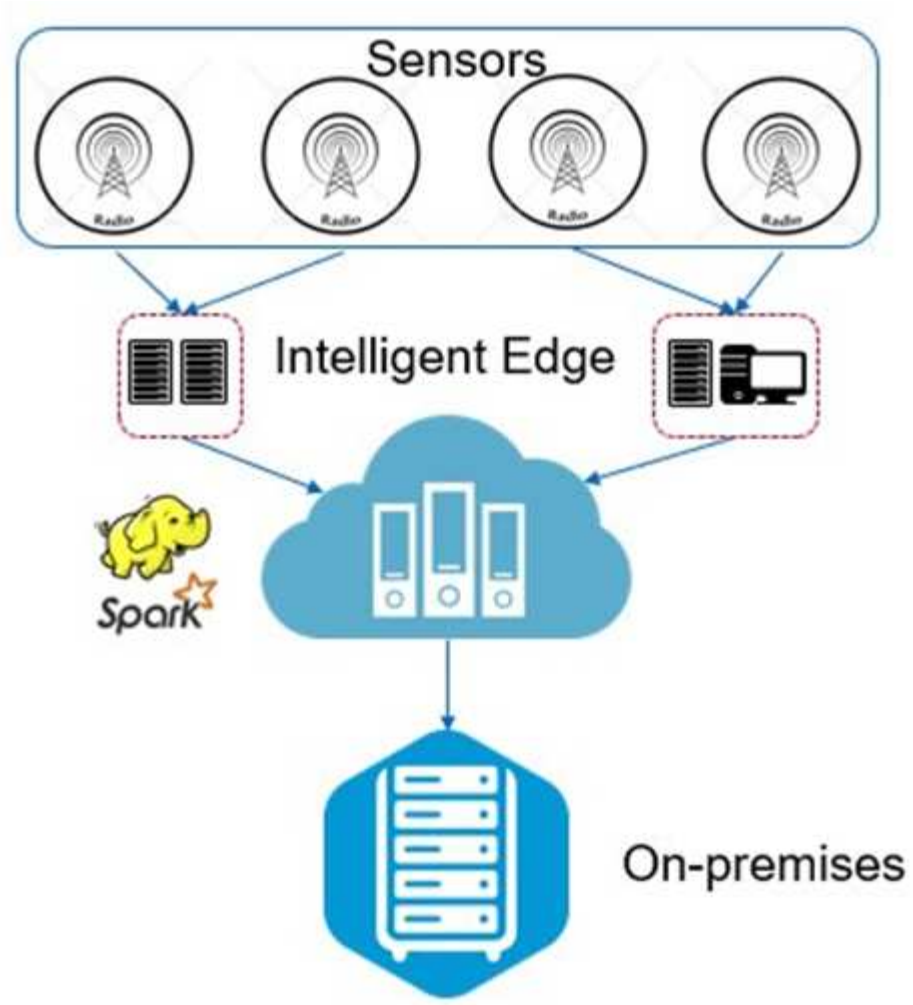
- 스냅샷 기술을 사용하여 공간 효율적인 백업을 생성할 수 있습니다.
- NetApp 툴을 통해 엔터프라이즈 관리 기능을 지원합니다.

"다음: 사용 사례 2 - 클라우드에서 사내로 백업 및 재해 복구"

사용 사례 2: 클라우드에서 사내로 백업 및 재해 복구

"이전: 활용 사례 1 - Hadoop 데이터 백업"

이 사용 사례는 아래 그림과 같이 클라우드 기반 분석 데이터를 사내 데이터 센터에 백업해야 하는 브로드캐스트 고객을 기반으로 합니다.



시나리오

이 시나리오에서는 IoT 센서 데이터가 클라우드로 수집되고 AWS 내의 오픈 소스 Apache Spark 클러스터를 사용하여 분석됩니다. 따라서 클라우드에서 처리된 데이터를 사내로 백업해야 합니다.

요구사항 및 당면 과제

이 사용 사례의 주요 요구사항과 과제는 다음과 같습니다.

- 데이터 보호를 사용하도록 설정하면 클라우드의 운영 Spark/Hadoop 클러스터에 성능 영향이 미치지 않습니다.
- 클라우드 센서 데이터를 효율적이고 안전한 방식으로 이동하고 사내로 보호해야 합니다.
- 온디맨드, 즉각적인, 낮은 클러스터 로드 시간 등 다양한 조건에서 데이터를 클라우드로 유연하게 전송할 수 있습니다.

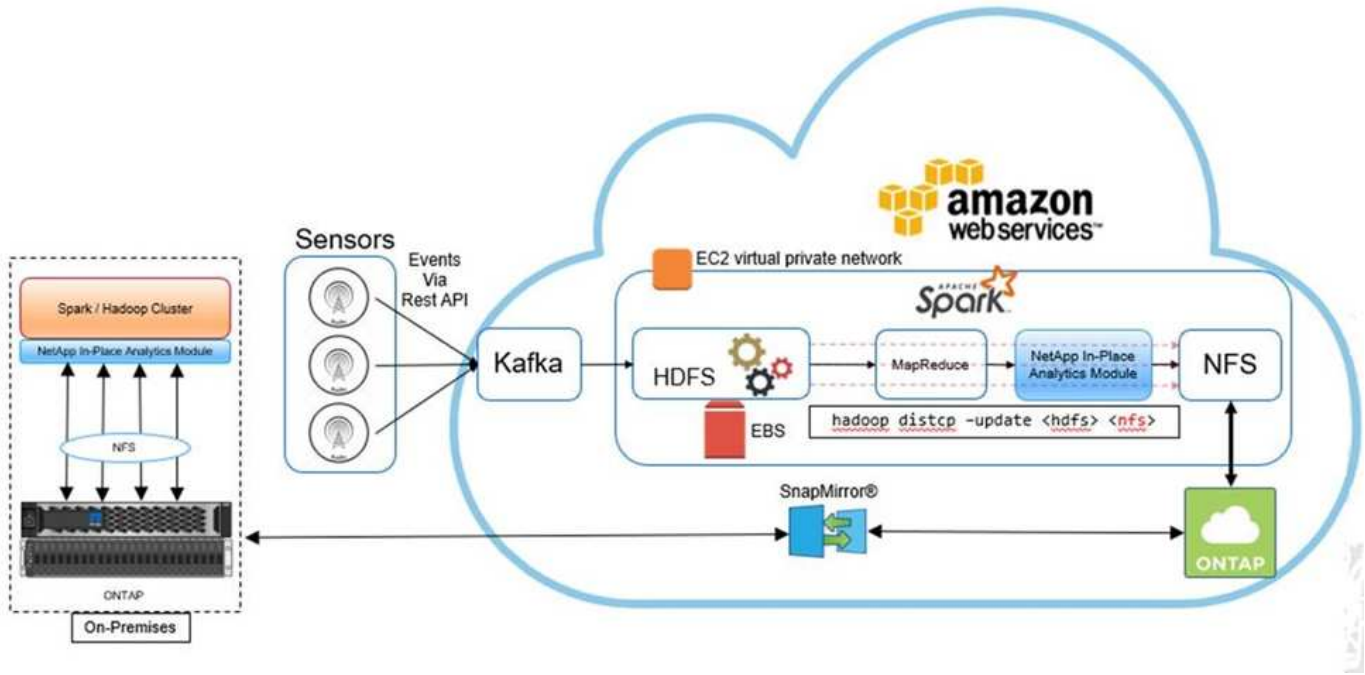
해결 방법

이 고객은 Spark 클러스터 HDFS 스토리지에 AWS EBS(Elastic Block Store)를 사용하여 Kafka를 통해 원격 센서에서 데이터를 수신 및 수집했습니다. 따라서 HDFS 스토리지는 백업 데이터의 소스 역할을 합니다.

이러한 요구사항을 충족하기 위해 NetApp ONTAP Cloud를 AWS에 구축하고 NFS 공유를 생성하여 Spark/Hadoop 클러스터의 백업 타겟 역할을 수행합니다.

NFS 공유가 생성된 후 데이터 이동 없는 분석 모듈을 활용하여 HDFS EBS 스토리지에서 ONTAP NFS 공유로 데이터를 복제합니다. ONTAP 클라우드의 NFS에 데이터가 상주한 후에는 SnapMirror 기술을 사용하여 필요에 따라 클라우드에서 사내 스토리지로 데이터를 안전하고 효율적으로 미러링할 수 있습니다.

이 이미지는 클라우드에서 사내 솔루션으로 백업 및 재해 복구를 보여줍니다.



"다음: 사용 사례 3 - 기존 Hadoop 데이터에 대해 DevTest를 설정합니다."

사용 사례 3: 기존 Hadoop 데이터에 대해 DevTest 활성화

"이전: 사용 사례 2 - 클라우드에서 사내로 백업 및 재해 복구"

이 사용 사례에서 고객은 DevTest와 보고 목적으로 동일한 데이터 센터와 원격 위치에서 대량의 분석 데이터를 포함하는 기존 Hadoop 클러스터를 기반으로 새로운 Hadoop/Spark 클러스터를 신속하고 효율적으로 구축해야 합니다.

시나리오

이 시나리오에서는 사내 및 재해 복구 위치에 대규모 Hadoop 데이터 레이크를 구축하고 Spark/Hadoop 클러스터를 여러 개 구축했습니다.

요구사항 및 당면 과제

이 사용 사례의 주요 요구사항과 과제는 다음과 같습니다.

- DevTest, QA 또는 동일한 운영 데이터에 액세스해야 하는 기타 목적으로 여러 Hadoop 클러스터를 생성합니다. 여기서 문제는 매우 큰 Hadoop 클러스터를 공간 효율적인 방식으로 여러 번 즉시 클론 복제해야 한다는 것입니다.
- 운영 효율성을 위해 Hadoop 데이터를 DevTest 및 보고 팀에 동기화합니다.
- 운영 클러스터와 새 클러스터 간에 동일한 자격 증명을 사용하여 Hadoop 데이터를 배포합니다.
- 예약된 정책을 사용하여 운영 클러스터에 영향을 주지 않고 QA 클러스터를 효율적으로 생성합니다.

해결 방법

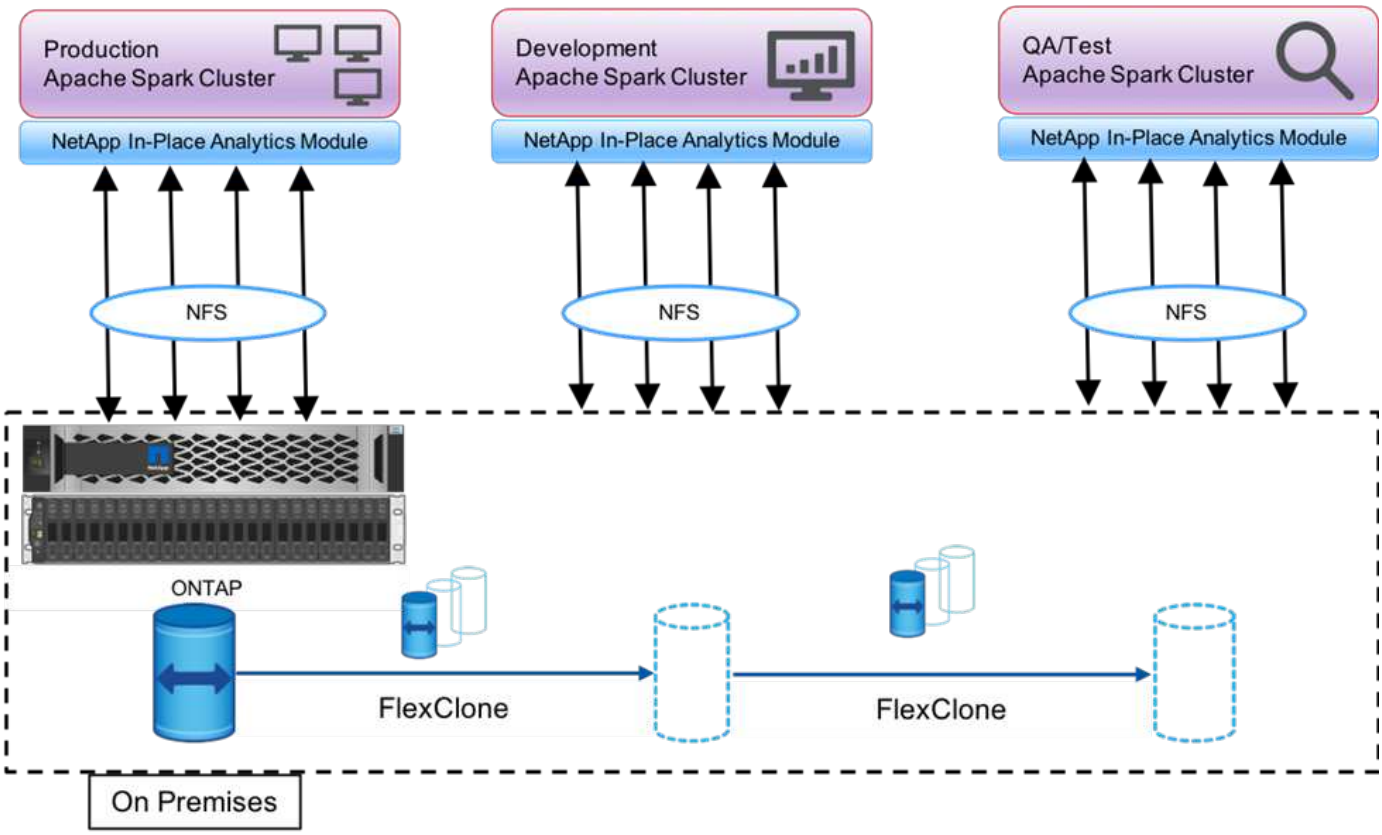
FlexClone 기술은 방금 설명한 요구 사항에 응답하는 데 사용됩니다. FlexClone 기술은 스냅샷 복사본의 읽기/쓰기 복사본입니다. 이 기능은 상위 스냅샷 복사본 데이터에서 데이터를 읽고 새 블록/수정된 블록에 대해 추가 공간만 사용합니다. 빠르고 공간 효율적입니다.

먼저, NetApp 일관성 그룹을 사용하여 기존 클러스터의 스냅샷 복사본을 생성했습니다.

NetApp System Manager 또는 스토리지 관리 프롬프트 내의 Snapshot 복사본 일관성 그룹 스냅샷 복사본은 애플리케이션 정합성이 보장된 그룹 스냅샷 복사본이며, FlexClone 볼륨은 일관성 그룹 스냅샷 복사본을 기반으로 생성됩니다. FlexClone 볼륨이 상위 볼륨의 NFS 엑스포트 정책을 상속한다는 점을 언급하는 것이 좋습니다. 스냅샷 복사본이 생성된 후에는 아래 그림과 같이 DevTest 및 보고를 위해 새 Hadoop 클러스터를 설치해야 합니다. In-Place Analytics Module은 NFS 데이터에 대한 In-Place Analytics Module 사용자 및 그룹 인증을 통해 새 Hadoop 클러스터에서 클론 생성된 NFS 볼륨에 액세스합니다.

적절한 액세스 권한을 가지려면 새 클러스터에 현재 위치 분석 모듈 사용자 및 그룹 구성에서 구성된 사용자에 대해 동일한 UID와 GUID가 있어야 합니다.

이 이미지는 DevTest용 Hadoop 클러스터를 보여 줍니다.



"다음: 사용 사례 4 - 데이터 보호 및 멀티 클라우드 연결"

사용 사례 4: 데이터 보호 및 멀티 클라우드 연결

"이전: 사용 사례 3 - 기존 Hadoop 데이터에 대해 DevTest 설정"

이 사용 사례는 고객의 빅데이터 분석 데이터를 위한 멀티 클라우드 연결을 제공하는 클라우드 서비스 파트너와 관련이 있습니다.

시나리오

이 시나리오에서는 여러 소스에서 AWS로 수신된 IoT 데이터가 NPS의 중앙 위치에 저장됩니다. NPS 스토리지는 AWS 및 Azure에 위치한 Spark/Hadoop 클러스터에 연결되므로, 여러 클라우드에서 실행되는 빅데이터 분석 애플리케이션이 동일한 데이터에 액세스할 수 있습니다.

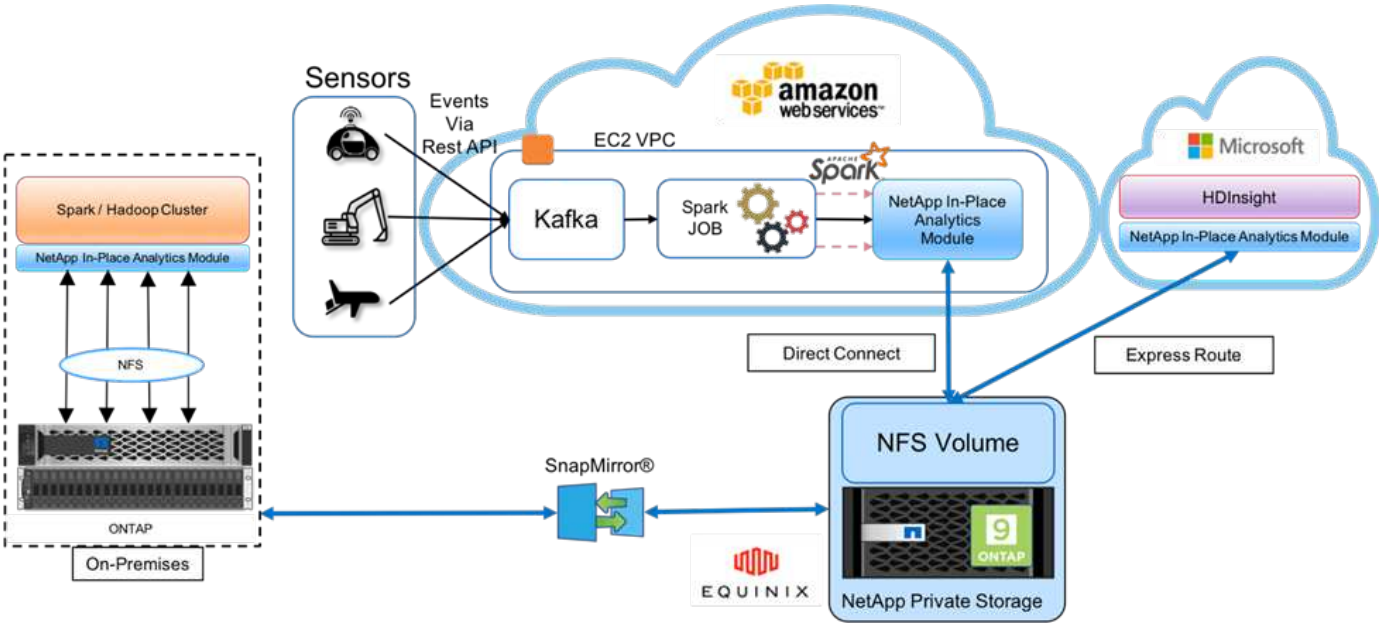
요구사항 및 당면 과제

이 사용 사례의 주요 요구사항과 과제는 다음과 같습니다.

- 고객은 여러 클라우드를 사용하여 동일한 데이터에 대한 분석 작업을 실행하려고 합니다.
- 다양한 센서와 허브를 통해 사내, 클라우드와 같은 다양한 소스로부터 데이터를 받아야 합니다.
- 솔루션은 효율적이고 비용 효율적이어야 합니다.
- 주요 과제는 사내 및 다른 클라우드 간에 하이브리드 분석 서비스를 제공하는 비용 효율적이고 효율적인 솔루션을 구축하는 것입니다.

해결 방법

이 이미지는 데이터 보호 및 멀티 클라우드 연결 솔루션을 보여줍니다.



위 그림과 같이 센서의 데이터가 스트리밍되어 Kafka를 통해 AWS Spark 클러스터로 수집됩니다. 이 데이터는 Equinix 데이터 센터 내의 클라우드 공급자 외부에 있는 NPS에 있는 NFS 공유에 저장됩니다. NetApp NPS는 Direct Connect와 Express Route 연결을 통해 Amazon AWS 및 Microsoft Azure에 연결되므로, 고객은 데이터 이동 없는 분석 모듈을 활용하여 Amazon 및 AWS 분석 클러스터 모두에서 데이터에 액세스할 수 있습니다. 이 접근 방식은 여러 하이퍼 스케일러의 클라우드 분석 문제를 해결합니다.

따라서 사내 스토리지와 NPS 스토리지 모두 ONTAP 소프트웨어를 실행하므로 SnapMirror는 사내 클러스터에 NPS 데이터를 미러링하여 사내 및 여러 클라우드에 하이브리드 클라우드 분석을 제공할 수 있습니다.

최적의 성능을 위해 일반적으로 여러 네트워크 인터페이스 및 직접 연결/빠른 경로를 사용하여 클라우드 인스턴스에서 데이터에 액세스할 것을 권장합니다.

"다음: 사용 사례 5 - 분석 워크로드 가속화"

사용 사례 **5:** 분석 워크로드 가속화

"이전: 사용 사례 4 - 데이터 보호 및 멀티 클라우드 연결"

이 시나리오에서는 NetApp NFS 스토리지 솔루션을 사용하여 대형 금융 서비스 및 투자 은행의 분석 플랫폼을 현대화함으로써 자산 관리 및 정량적 사업부의 투자 위험 및 파생물 분석에 상당한 개선을 이루었습니다.

시나리오

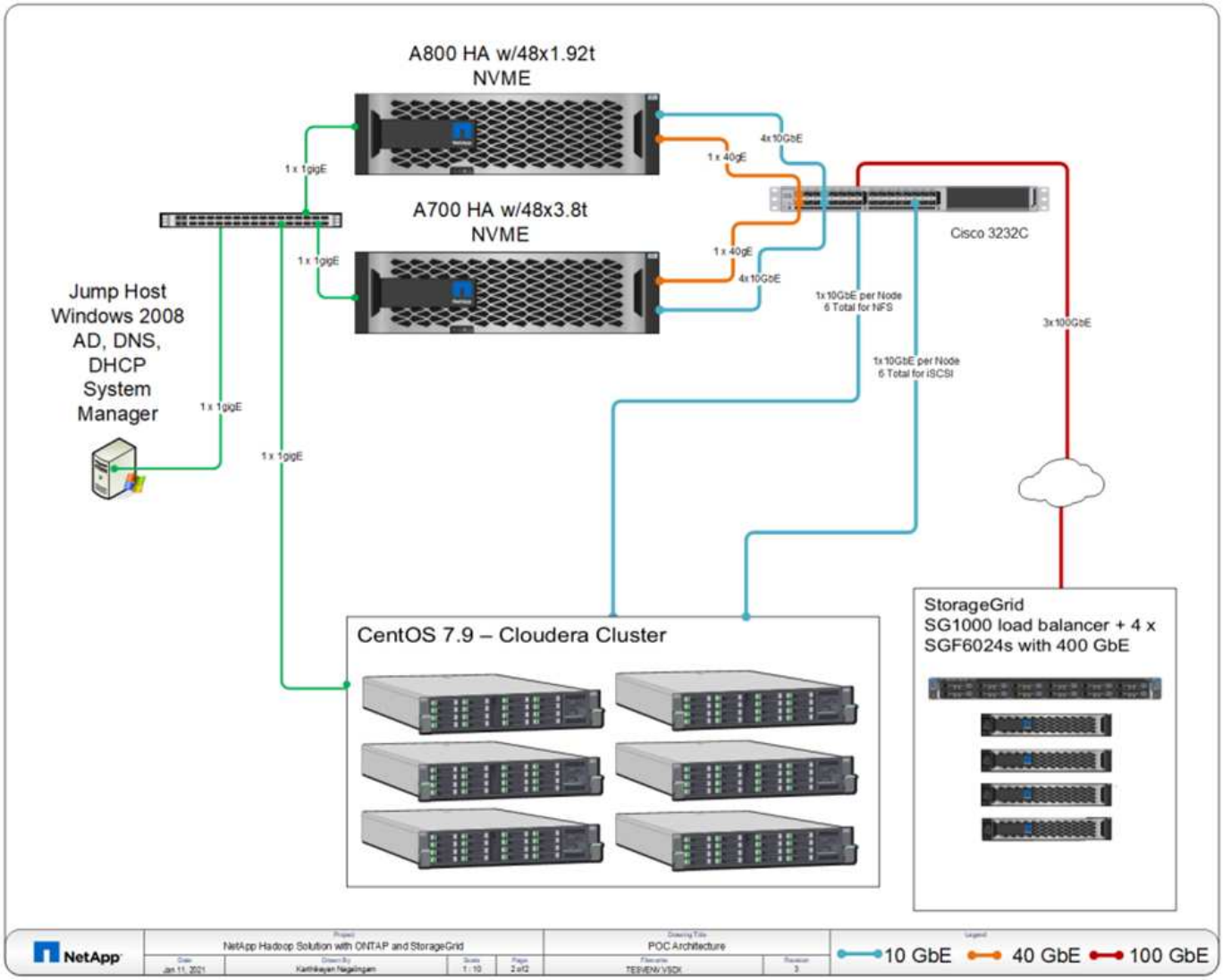
고객의 기존 환경에서 분석 플랫폼에 사용되는 Hadoop 인프라는 Hadoop 서버의 내부 스토리지를 활용했습니다. JBOD 환경의 독점 특성으로 인해 조직 내의 많은 내부 고객은 실시간 데이터의 반복 샘플에 의존하는 시뮬레이션인 Monte Carlo 정량 모델을 활용할 수 없었습니다. 시장 이동이 불확실성의 영향을 이해하는 데 있어 최적화되지 않은 능력은 정량적 자산 관리 사업부에 비호의적으로 작용했습니다.

요구사항 및 당면 과제

이 은행의 정량적 사업부는 정확하고 시기 적절한 예측을 얻기 위한 효율적인 예측 방법을 원했습니다. 이를 위해 IT 팀에서는 인프라를 현대화하고, 기존 I/O 대기 시간을 줄이며, Hadoop, Spark와 같은 분석 애플리케이션의 성능을 개선하여 투자 모델을 효율적으로 시뮬레이션하고, 잠재적인 이익을 측정하고, 위험을 분석해야 한다는 사실을 깨달았습니다.

해결 방법

고객은 기존 Spark 솔루션에 대한 JBOD를 가지고 있었습니다. 그런 다음 NetApp ONTAP, NetApp StorageGRID 및 MinIO Gateway to NFS를 활용하여 잠재적인 이익과 위험을 평가하는 투자 모델에 대한 시뮬레이션 및 분석을 실행하는 금융 그룹의 정량적 대기 시간을 줄였습니다. 이 이미지는 NetApp 스토리지의 Spark 솔루션을 보여줍니다.



위 그림과 같이 StorageGRID A800, A700 시스템 및 AFF는 Spark가 있는 6노드 Hadoop 클러스터 및 데이터 분석 작업을 위한 YARN 및 Hive 메타데이터 서비스의 NFS 및 S3 프로토콜을 통해 쪽모이 세공 파일에 액세스하기 위해 구축되었습니다.

고객의 기존 환경에서 DAS(직접 연결 스토리지) 솔루션을 사용할 경우 컴퓨팅과 스토리지를 독립적으로 확장한다는 단점이 있었습니다. Spark용 NetApp ONTAP 솔루션을 통해 은행의 재무 분석 사업부에서 스토리지를 컴퓨팅에서 분리하여 필요에 따라 인프라 리소스를 보다 효율적으로 제공할 수 있게 되었습니다.

NFS와 함께 ONTAP를 사용함으로써 컴퓨팅 서버 CPU는 Spark SQL 작업에 거의 전적으로 활용되었고 I/O 대기 시간이 약 70% 감소되어 스파크 워크로드에 더 나은 컴퓨팅 성능과 성능 향상을 제공했습니다. 또한 CPU 활용률을 높임으로써 고객이 GPUDirect와 같은 GPU를 활용하여 플랫폼을 더욱 현대화할 수 있도록 했습니다. 또한 StorageGRID는 스파크 워크로드를 위한 저렴한 스토리지 옵션을 제공하며 MinIO 게이트웨이는 S3 프로토콜을 통해 NFS 데이터에 대한 안전한 액세스를 제공합니다. 클라우드의 데이터에 대해 NetApp은 Cloud Volumes ONTAP, Azure NetApp Files 및 NetApp Cloud Volumes Service를 권장합니다.

"다음: 결론."

결론

"이전: 사용 사례 5 - 분석 워크로드 가속화"

이 섹션에서는 다양한 Hadoop 데이터 보호 요구사항을 충족하기 위해 NetApp에서 제공하는 사용 사례 및 솔루션을 요약합니다. 고객은 NetApp이 제공하는 Data Fabric을 사용하여 다음을 수행할 수 있습니다.

- NetApp의 풍부한 데이터 관리 기능과 Hadoop 기본 워크플로우와의 통합을 활용하여 적합한 데이터 보호 솔루션을 유연하게 선택할 수 있습니다.
- Hadoop 클러스터의 백업 윈도우 시간을 약 70% 단축
- Hadoop 클러스터 백업으로 인한 성능 영향을 없앱니다.
- 여러 클라우드 공급자로부터 데이터 보호 및 데이터 액세스를 동시에 단일 분석 데이터 소스에 제공
- FlexClone ® 기술을 사용하여 빠르고 공간 효율적인 Hadoop 클러스터 복사본을 생성합니다.

추가 정보를 찾을 수 있는 위치

이 문서에 설명된 정보에 대한 자세한 내용은 다음 문서 및/또는 웹 사이트를 참조하십시오.

- NetApp 빅데이터 분석 솔루션

  "https://www.netapp.com/us/solutions/applications/big-data-analytics/index.aspx"

- NetApp 스토리지를 사용하는 Apache Spark 워크로드

  https://www.netapp.com/pdf.html?item=/media/26877-nva-1157-deploy.pdf

- Apache Spark용 NetApp 스토리지 솔루션

  "https://www.netapp.com/media/16864-tr-4570.pdf"

- NetApp에서 지원하는 Data Fabric 기반 Apache Hadoop

  "https://www.netapp.com/media/16877-tr-4529.pdf"

- NetApp 데이터 이동 없는 분석 모듈

  "https://library.netapp.com/ecm/ecm_download_file/ECMLP2854071"

감사의 말

- Paul Burland, 영업 담당자, ANZ Victoria District Sales, NetApp
- Hoseb Dermanilian, NetApp 비즈니스 개발 매니저
- Lee Dorrier, NetApp MPSG 이사
- David Thiessen, 시스템 엔지니어, ANZ Victoria District SE, NetApp

버전 기록

버전	날짜	문서 버전 기록
버전 1.0	2018년 1월	최초 릴리스
버전 2.0	2021년 10월	사용 사례 5로 업데이트: 분석 워크로드 가속화

Copyright Information

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

Trademark Information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.